



Optimering av BFP-algoritm för komprimering och avkomprimering

Optimering med GPU hårdvara

Optimization of BFP-algorithm for Compression and Decompression
Optimization with GPU hardware

Sami Al Chalati
Martin Forsbacke

Fakulteten för hälsa, natur-och teknikvetenskap

Datavetenskap

C-uppsats 15hp

Handledare: Per Hurtig, Bengt Hallinger och Thomas Svadling

Examinator: Johan Eklund

Datum: 2025-05-30

Förord

Tack till våra föräldrar, KAU samt alla bra lärare och de handledare som har hjälpt oss och coachat oss att göra denna uppsats.

Sammanfattning

Mobilnätverk och telekom är områden som används dagligen av majoriteten av befolkningen. Tillgång till dessa tjänster är avgörande för flera samhällssektorer, såsom industri, sjukvård och civil infrastruktur. 5G-celler kräver stora datamängder vid kommunikation med mobila enheter. För att minska belastningen på bandbredden används komprimering, vilket reducerar mängden data som behöver föras över länken mellan radioenhet och basstation.

Huvudsyftet med detta arbete är att optimera Block Floating Point (BFP)-algoritmen med hjälp av Graphic Processing Unit (GPU) och undersöka om Nvidia-grafikkort utgör en rimlig plattform för att exekvera BFP-algoritmen. Det andra målet är att utveckla ett program som uppfyller de tidskrav som ställs för två frekvensband: lågband och midband.

Innan själva utvecklingen analyserades pseudokoden för BFP-algoritmen för att identifiera vilka delar som passar för parallell exekvering. Under arbetets gång användes verktyg och tidsmätningar för att utvärdera prestandan av mjukvaran vid exekvering.

Resultaten visar att exekvering av BFP-algoritmen med GPU inte är en optimal lösning. Detta beror främst på att själva algoritmen exekveras snabbare än den tid det tar att överföra data till och från GPU, vilket leder till ett ineffektivt utnyttjande av hårdvaran.

Abstract

Mobile networks and telecommunications are areas used daily by the majority of the population. Access to these services is crucial for several sectors of society, such as industry, healthcare, and civil infrastructure. 5G cells require large amounts of data when communicating with mobile devices. To reduce the load on bandwidth, compression is used, which reduces the amount of data that needs to be transmitted over the link between the radio unit and the base station. The main purpose of this work is to optimize the BFP algorithm using a GPU and to investigate whether Nvidia graphics cards constitute a reasonable platform for executing the BFP algorithm. The second goal is to develop a program that meets the timing requirements for two frequency bands: low band and mid band. Before the actual development, the pseudocode for the BFP algorithm was analyzed to identify which parts are suitable for parallel execution. During the course of the work, tools and timing measurements were used to evaluate the software's performance during execution. The results show that executing the BFP algorithm using a GPU is not an optimal solution. This is mainly because the algorithm itself executes faster than the time it takes to transfer data to and from the GPU, which leads to inefficient utilization of the hardware.

Innehåll

Förord	i
Sammanfattning	iii
Abstract	v
Figurer	xii
Tabeller	xiii
Listings	xv
1 Introduktion	1
1.1 Bakgrund	2
1.2 Problembeskrivning	3
1.3 Syfte och mål	3
1.4 Etik och samhälle	4
1.5 Metod	4
1.6 Uppdragsgivare	5
1.7 Fördelning av arbete	5
1.8 Avgränsningar	5
1.9 Disposition	6

2	Bakgrund	7
2.1	Open Radio Access Network (O-RAN)	7
2.2	Quadrature Amplitude Modulation (QAM)	8
2.2.1	Olika typer av QAM	10
2.3	BFP algoritm	14
2.3.1	GPU vs CPU	16
2.4	CUDA	17
2.4.1	Anledningen till att använda CUDA	18
2.4.2	Arbetsflödet i CUDA	18
2.4.3	Att programmera med CUDA i C/C++:	18
2.4.4	Minnesallokering i CUDA	20
2.4.5	Antal block och trådar i CUDA	20
2.5	Metoder	21
2.6	Liknande arbeten	22
2.6.1	Block-skaland kompression	23
2.6.2	μ - Law kompression	23
2.6.3	Vektoraddition projekt med parallellisering	24
2.6.4	Kernel-exekvering (starta CUDA kärnor)	24
2.7	Optimering	27
3	Design	29
3.1	Problembeskrivning	29
3.1.1	Hur nuvarande system fungerar	30
3.1.2	Lågband och midband	30
3.2	Strategier för att minska exekveringstider	31
3.2.1	Planen för att lösa problemet	32
3.3	Lösningen på designnivå	33
3.4	Verifiering av lösningen	34

3.4.1	Hårdvara	35
3.4.2	Mjukvara	36
3.4.3	Parametrar	36
4	Implementation	39
4.1	Implementation	40
4.2	C variant- Referensprogram	40
4.3	CUDA	45
4.4	Seriell exekvering	46
4.5	Parallell exekvering i ett PRB-block	46
4.6	Parallel exekvering över alla PRB block	48
4.7	En tråd per element	49
5	Resultat	53
5.1	Mål och resultat	53
5.2	CUDA program	57
5.2.1	Exekvering på en kärna	58
5.2.2	Parallell exekvering internt i ett block	58
5.2.3	Parallell exekvering över alla block	61
5.2.4	Parallell exekvering med en tråd för varje element	61
5.3	Utvärdering	68
6	Slutsats	71
6.1	Diskussion	71
6.2	Sammanfattning	72
6.3	Utvärdering	72
6.4	Vidare arbete	73
	Litteraturförteckning	74

Bilagor	78
A Bilagor	81
B Listningar	85

Figurer

2.1	Radiokommunikationsnätverk med radioenheter RU och DU.[1]	9
2.2	Exempel på IQ modulation [2]	10
2.3	QAM konstellationsgraf[3]	12
2.4	Fler exempel på konstellationsgrafer [4]	13
2.5	Illustration av skillnaden i processorarkitektur mellan CPU och GPU [5]	17
2.6	Diagram på Block scaling algoritm [6]	22
2.7	Diagram på μ -law algoritm [6]	23
3.1	Data för I och Q i ett PRB block [6]	31
3.2	Programmets design och exekvering process	37
5.1	Stapeldiagram över exekveringstiderna för programversionerna (tider per symbol på midband på RTX3090).	55
5.2	Stapeldiagram över tiderna på programmen använt under arbetet(tider för en symbol på lågband på RTX3090)	56
5.3	Stapeldiagram över komprimering för en symbol på CPU (en symbol)	57
5.4	Stapeldiagram för seriell exekvering för GPU (en symbol)	59
5.5	Stapeldiagram för parallel exekvering intärnt i ett PRB med GPU (en symbol)	60
5.6	Stapeldiagram för parallel exekvering över alla block på GPU (en symbol)	62
5.7	Stapeldiagram över komprimering för en tidsslot på GPU.	63

5.8	Nsight system mätning över 1 stream	64
5.9	Nsight system mätning över 2 stream	65
5.10	Nsight system mätning över 14 stream	66
5.11	Stapeldiagram över körningar för olika antal celler.	67
5.12	Nsight system exempel på tider i en GPU körning	68
A.1	Visuell bild över program utan extra strömmar	82
A.2	Visuell bild över program med extra strömmar	83

Tabeller

3.1	Tabell över de mjukvaror vi utvecklade under arbetet och diskuterar i rapporten	33
3.2	Hårdvaruspecifikationer för gamla utvecklingsservern	35
3.3	Uppdaterade hårdvaruspecifikationer	35
5.1	Tabell över programmen tiderna för en tidsslot, en symbol och per antenn (gamla servern)	69
5.2	Tabell över programmen tiderna för en tidsslot, en symbol och per antenn (nya servern)	70

Lista över listningar

2.1	Psuedokod för PFB-Komprimering	15
2.2	Psuedokod för PFB-Avkodning	16
2.3	CPU exempel på seriell vektoraddition [7]	19
2.4	GPU exempel på seriell vektoraddition [7]	19
2.5	Minnesallokering i CUDA [7]	20
2.6	Exempel på Kernelanrop i CUDA [7]	21
2.7	CUDA kod exempel på vektoraddition med trådar	27
4.1	Kodvariant för BFP på CPU seriellt	45
4.2	Exempel på trådhanteringen inom CUDA	45
4.3	Exempel på GPU seriell exekvering för dekomprimera	46
4.4	Kodexempel på Warp divergent	47
4.5	GPU parallell inuti ett PRB dekomprimering exempel	48
4.6	Exempel på parallell komprimering på GPU block	49
4.7	GPU exempel på minnesallokering	50
4.8	Kodexempel på hur man kan använda flera streaming multiprocessors .	51
B.1	Seriell exekvering på GPU	88
B.2	GPU parallell i ett PRB	91
B.3	GPU parallell över PRB	94
B.4	GPU helt parallell	99

Akronymer

AI Artificial Intelligence. 18

AM Amplitude Modulation. 8

API Application Programming Interface. 18, 22

BFP Block Floating Point. iii, v, 1, 3–7, 14, 16, 22, 23, 29, 30, 32, 34, 40, 53, 66, 67, 71–73

BPSK Binary Phase Shift Keying. 3, 9

CPU Central Processing Unit. 2, 3, 5, 6, 16–19, 29, 30, 32, 34, 39, 40, 45, 53, 58, 71, 72

CUDA Compute Unified Device Architecture. 2, 4–7, 17–22, 24, 27, 31, 32, 40, 45, 46, 48–50, 53, 57, 58, 65, 72

DU Digital Unit. 2, 3, 7, 8

GPU Graphic Processing Unit. iii, v, xii, 2–6, 16–21, 27–32, 34, 39, 40, 45, 46, 53, 54, 57, 58, 60, 61, 64–68, 71–73, 81

IQ In phase Quadrature. 10, 23

OFDM Ortogonal Frequency Division Multiplexing. 3

O-RAN Open Radio Access Network. 1, 2, 6, 7, 30

PCIe Peripheral Component Interconnect Express. 6, 33, 58, 64, 68, 73

PRB Physical Resource Block. 14, 23, 30, 31, 33, 34, 39, 40, 46, 48, 58, 61, 66, 81

QAM Quadrature Amplitude Modulation. 3, 7–11, 14

RTT Round Trip Time. 64

RTX Ray Tracing eXtreme. 5, 68, 73

RU Radio Unit. 2, 3, 7, 8

SM Streaming Multiprocessor. 21, 28, 36, 49, 50, 63, 65, 66, 68, 69, 72, 81

VS Code Visual Studio Code. 36

Kapitel 1

Introduktion

Moderna mobila enheter är beroende av snabba, stabila nätverk för att möjliggöra tillgång till internetjänster. För att uppfylla dessa krav använder dagens mobilnätverk avancerade teknologier som erbjuder höghastighetsuppkoppling, låg latens och sömlös övergång mellan nätverk. Fjärde och femte generationens mobilnätverk (4G och 5G) utgör grunden för den nuvarande infrastrukturen, där särskilt 5G medför förbättrad stabilitet, reducerad paketförlust samt ökad kapacitet både i datamängd och överföringskvalitet. Arbetet med att vidareutveckla dessa tekniker pågår fortlöpande, och en ny standard för 6G är för närvarande under utveckling.

5G-nätverksarkitekturen är uppbyggd av flera samverkande komponenter som möjliggör funktionalitet inom trådlös kommunikation. En central del i denna arkitektur är Open Radio Access Network (O-RAN) vilket definierar hur olika radiokomponenter kan integreras och användas för effektiv dataöverföring.

Detta arbete fokuserar på en specifik del av O-RAN, nämligen komprimeringsalgoritmer för reducering av datastorlek. En av de algoritmer som studeras är BFP vilken används för att komprimera data mellan basstation och radioenhet. Genom att minska datamängden reduceras kraven på bandbredd, vilket är avgörande för att upprätthålla prestanda i högbelastade nätverk.

För att förbättra algoritmens exekveringseffektivitet tillämpas Compute Unified Device Architecture (CUDA)-programmering [5], en plattform och ett API från NVIDIA som möjliggör parallell bearbetning med hjälp av GPU-acceleration. Arbetet demonstrerar även skillnaderna mellan Central Processing Unit (CPU)- och GPU-arkitektur vad gäller processhantering och prestanda, särskilt vid parallella beräkningar.

Utöver teoretisk genomgång genomförs även praktiska experiment för att utvärdera GPUns potential i beräkningsintensiva miljöer. De verktyg som används i arbetet inkluderar CUDA, C/C++, GitLab samt analysverktyget Nsight Systems. En översikt över den använda hårdvaran finns i Avsnitt 3.4.1.

1.1 Bakgrund

Telekomföretag levererar kommunikationstjänster som mobiltelefoni, internetuppkoppling och övrig dataöverföring. Telekomföretag bygger och underhåller nätverk genom att använda infrastruktur som celltorn, fiberoptik och kommunikationssatelliter. O-RAN används av telekomföretag, vilket förenklar infrastrukturombyggnadsprocessen av konstruktion av infrastruktur då O-RAN är ett standardiserat och öppet ramverk. O-RAN möjliggör införskaffning och tillverkning av komponenter när RAN finns tillgänglig för verksamheterna. Ett exempel på en överföring är mellan två mobiltelefoner. Olika system och mjukvaror är avsedda för att reducera kostnaderna och öka flexibiliteten vid dataöverföring. Genom att införa O-RAN kan telekomföretag förbättra nätverkseffektiviteten och överföra stora mängder data snabbt eftersom O-RAN delar upp nätverket i flexibla delar Digital Unit (DU) och Radio Unit (RU). När data skickas mellan två slutanvändare sker detta via mobilnätverket som använder en algoritm för att komprimera informationen, som sedan konverteras till radiovågor och överförs till den andra enheten. En vanlig metod som telekomföretag använder är digital modulering som används i telekommunikation för att konvertera digital data till radiosignaler. Radiosignaler kan

representeras matematiskt med komplexa tal i form av sinus- och cosinuskurvor. Det bör nämnas att det finns olika varianter av digital modulering. Den som används i detta arbete är Quadrature Amplitude Modulation (QAM)[2]. QAM är en metod som kombinerar både amplitud- och fasmodulation för att överföra flera bitar per subcarrier som är en enskild frekvenskanal som bär data i Ortogonal Frequency Division Multiplexing (OFDM). Det finns flera nivåer av QAM(4,8,16,64, 256), de högre nivåerna tillåter högre dataström jämfört med de lägre och Binary Phase Shift Keying (BPSK) som även kan kallas QAM 2, som endast kodar 2 bitar per subcarrier. [8]. BPSK och QAM tas upp mer i detalj i kapitel 2.2.1. OFDM överför subcarrier med hjälp av denna modulering via radiovågor på en specifik frekvens i den tillgängliga bandbredden. OFDM är en teknik som gör att de olika signalerna som överförs inte kommer att störa varandra och orsaka brus i radiovågorna.

1.2 Problembeskrivning

Inom telekomsystem komprimeras QAM-data mellan en DU och en RU-enhet med hjälp av BFP-algoritmen, som i detta fall exekveras sekventiellt på en CPU genom användning av en enskild kärna. Problemet som ska undersökas är huruvida det är fördelaktigt att i stället köra algoritmen på en GPU, och om algoritmen kan exekveras där med möjlighet att utnyttja GPU kapacitet för parallell bearbetning.

1.3 Syfte och mål

Huvudmålet är att optimera BFP-algoritmen genom parallell exekvering. Det innebär att programmet fördelar uppgifter mellan flera processorkärnor, vilket minskar belastningen på varje enskild kärna. Ett förenklat exempel: att koka tre ägg i en kastrull samtidigt tar mindre tid än att koka ett ägg i taget. Alltså får man mer gjort på samma tid.

Algoritmens exekvering ska optimeras för att uppfylla specifika prestandakrav. I denna rapport beaktas två olika tidsgränser, vilka representerar skilda frekvensbandsområden. Det första området, benämnt lågband, kännetecknas av lägre krav både avseende exekveringstid och datavolym. Det andra området, kallat midband, ställer däremot högre krav i form av kortare exekveringstid samt hantering av större datamängder.

Syftet med arbetet är att undersöka genomförbarheten i att använda GPU-acceleration för BFP-algoritmen i en basstationsmiljö, med särskilt fokus på de ovan nämnda kraven gällande exekveringstid och datavolym.

1.4 Etik och samhälle

Detta projekt behandlar inte någon direkt etisk eller samhällelig frågeställning, eftersom det enbart fokuserar på algoritmoptimering. Arbetet innebär inte utveckling av en ny applikation som kan ha en direkt påverkan på samhället eller andra tillämpningsområden.

1.5 Metod

Utvecklingen skedde genom att iterativt förbättra och testa. Detta skedde genom att studera tidigare CUDA-kod för att förstå hur de fungerar och titta på tutorials, implementera de principer, testa funktionaliteten och ta tider som programmet tog för att utföra arbetet. Efter varje iteration analyserades programmets beteende för att identifiera flaskhalsar och möjligheter till prestandaförbättring. Genom denna iterativa metod identifierades de faktorer som påverkar effektiviteten. Analyseringen skedde med hjälp av ett program som kallas Nsight System [9]. Nsight System används av utvecklare för att analysera och förbättra prestanda för GPU-program som använder CUDA.

1.6 Uppdragsgivare

Uppdragsgivaren är företaget Tietoevry, som är inriktade mot telekommunikation, bank, utbildning, energi, pappersindustrin, sjukvård samt ett par industrier till. Tietoevry bygger och utvecklar teknologi som 5G, uppkopplade bilar, smarta enheter och molnplattformar. De jobbar internationellt för många olika klienter.

Detta arbete är av intresse för Tietoevry eftersom undersökningen av CUDA och GPU är värdefull för att kartlägga återvändsgränder för framtida användningsområden. Teknologin kan användas för att avlasta CPU, vilket möjliggör beräkningar på kortare tid. Detta kan vara relevant för 6G, eftersom BFP algoritmen nuvarande körs seriellt på CPU och kan behöva snabbas upp.

1.7 Fördelning av arbete

Arbetet utfördes av Sami Al Chalati och Martin Forsbacke, där båda deltog i både mjukvaruutveckling och skrivande. När det gällde kodningen arbetade de tillsammans under hela processen. Vid skrivandet ansvarade varje student för en specifik sektion, men de granskade sedan varandras texter för att dubbelkolla språk och grammatik.

1.8 Avgränsningar

För att utvärdera BFP-algoritmens prestanda, samt beteende, när den exekveras i en parallelliserad miljö så krävs tydliga avgränsningar, eftersom vi inte har tillgång till obegränsade resurser eller tid. Undersökningen fokuserar därför uteslutande på att mäta och jämföra exekveringstider för BFP-algoritmen i tre olika scenarier: sekventiell körning på CPU, sekventiell körning på GPU samt flera parallella körningar på GPU. I projektet används Nvidias grafikkort Ray Tracing eXtreme (RTX) 3090 och RTX 5090, vilka skiljer sig åt vad gäller antal kärnor samt klockfrekvenser för både minne och

kärnor. Även Peripheral Component Interconnect Express (PCIe), som är den standardiserade anslutningen mellan moderkortet och grafikkortet, har en betydande inverkan på dataöverföringshastigheten mellan CPU och GPU [10]. För denna studie begränsar vi oss därför till att använda PCIe 4.0 x16 och PCIe 5.0 x16. Hårdvarans specifikationer har direkt påverkan på algoritmens prestanda, och med framtida generationer av RTX som potentiellt stödjer PCIe 6.0 med dubbelt så hög överföringshastighet kan latensen i kommunikationen mellan CPU och GPU reduceras ytterligare, vilket i sin tur skulle kunna förbättra den totala prestandan [11]. Noterbart är att denna studie inte omfattar mätning eller analys av hårdvarans energiförbrukning vid exekvering av algoritmerna.

1.9 Disposition

Den här rapporten är strukturerad enligt följande:

- Kapitel 2 innehåller en detaljerad bakgrund. Där beskrivs telekomsystem i O-RAN och BFP algoritmen samt hur CUDA används för att köra applikationer parallellt.
- Kapitel 3 beskriver hur vi planerade att skapa programmet, vilken mjukvara och hårdvara som användes i arbetet. Vilka förbättringar vi planerar att göra för att få resultatet.
- Kapitel 4 handlar om den tekniska implementationen. Det vill säga arbetet som utfördes inom optimering med parallell exekvering inom CUDA.
- Kapitel 5 behandlar den programvara som utvecklats under arbetets gång och relaterar resultaten från implementationen till de mål och syften som formulerats i problembeskrivningen
- Kapitel 6 sammanfattar resultatet, presenterar slutsatserna samt en tolkning av rimligheten att använda hårdvaran från arbetet.

Kapitel 2

Bakgrund

I detta kapitel presenteras de huvudkoncept som är relevanta för projektet. Först beskrivs O-RAN som utgör grunden för den radioteknik som används. Därefter beskrivs tekniken QAM i detalj samt hur den används för dataöverföring. Detta följs av BFP-algoritmen för komprimering och dekomprimering, samt en introduktion till CUDA. De metoder som användes och hur projektet genomfördes. Slutligen redogörs för liknande algoritmer som används inom O-RAN.

2.1 Open Radio Access Network (O-RAN)

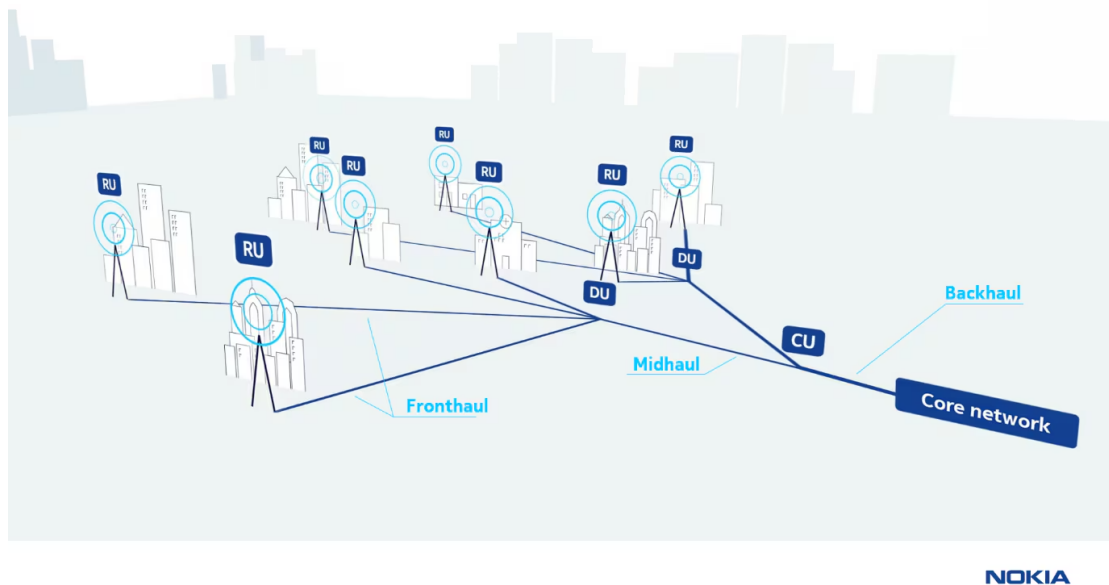
O-RAN är en specifikation över hur radionätverk (RAN) kan implementeras med generella radio-komponenter. Målet är att telekomföretag världen runt ska kunna implementera och bygga ut radionätet på ett enkelt sätt[1].

I ett nätverk, används en uppdelad nätverksarkitektur som kallas för stack. Den består av flera lager och funktioner som arbetar tillsammans för att leverera både kontrollsignaler (som styr uppkopplingar) och användardata (t.ex. surf och samtal). Två viktiga komponenter i denna stack är DU och RU. DU är en nätverksnod som behandlar data nära användaren. Den ansvarar för realtidsbearbetning, till exempel schemalägg-

ning av användare och hantering av radiokanaler. RU, å andra sidan, är den del av systemet som är kopplad direkt till antennerna och ansvarar för att sända och ta emot radiosignaler. Dessa två delar kommunicerar med varandra, och för att göra det effektivt vill man komprimera datan mellan dem för att spara bandbredden. DU ansvarar för att hantera realtidsbehandlingen av de lägre lagren i protokollstacken, särskilt lagren Medium Access Control (MAC) och Radio Link Control (RLC). Den spelar en avgörande roll i att hantera datalänken mellan användarens enhet och nätverket, vilket säkerställer att data överförs och tas emot effektivt. Arbetets fokus ligger på dataöverföringen mellan RU och DU. En DU eller basstation kommer ta data ifrån nätet och överföra dessa till sina radioantennerna eller RU. Figur 2.1 illustrerar ett radiokommunikationsnätverk med dess centrala komponenter: RU och DU som samverkar för att möjliggöra datakommunikation. Strukturen demonstrerar dataflödet från radiogränssnittet till nätverkskärnan. Det som vi ska komprimera är data som skickas mellan dessa för att reducera den nödvändiga bandbredden som krävs för överföringen. Data överförs mellan dessa enheter med hjälp av en teknik som kallas QAM.

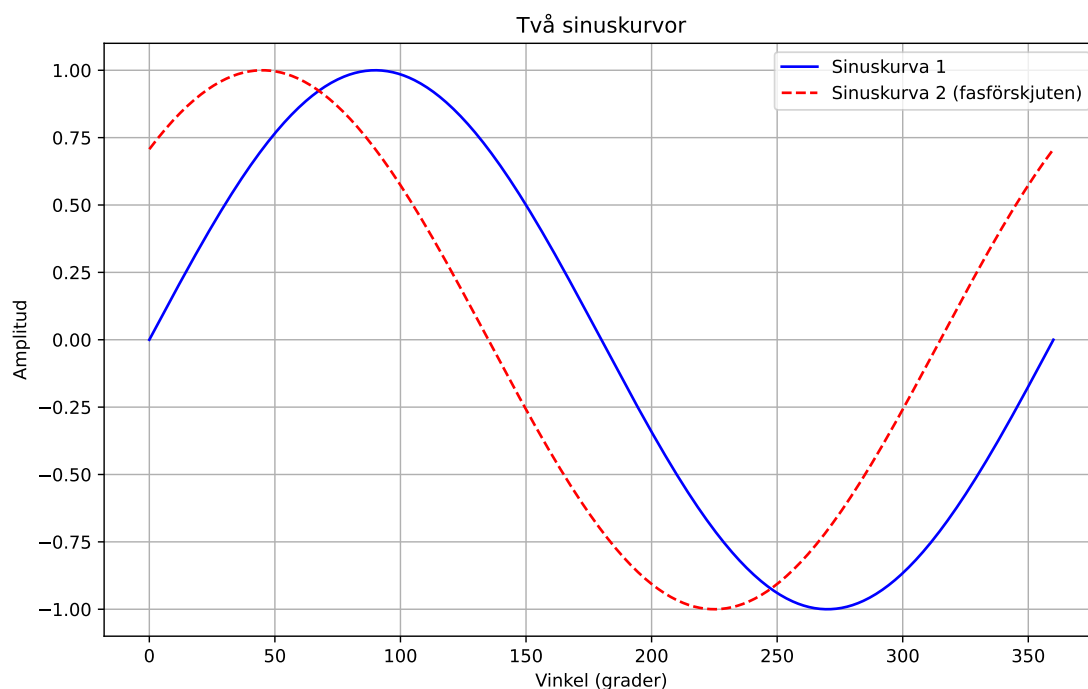
2.2 Quadrature Amplitude Modulation (QAM)

QAM är en teknik som bygger på att kombinera Amplitudmodulering Amplitude Modulation (AM) signaler till en enda kanal. Denna approach syftar till att fördubbla den effektiva bandbredden genom att utnyttja både amplitud- och fasvariationer för att överföra mer information per tidsenhet. För att överföra data på ett sömlöst sätt, används en QAM-modulator som används för att översätta digitala paket till en analog signal. QAM används för att uppnå höga nivåer av spektrumanvändning. QAM-signalen består av en subcarrier som moduleras med två komponenter: en I-signal och en Q-signal, som är sinussignaler med 90° fasförskjutning. Denna fasförskjutning gör att I-signalen vanligtvis representeras som en cosinusvåg, medan Q-signalen representeras som en



Figur 2.1: Radiokommunikationsnätverk med radioenheter RU och DU.[1]

sinusvåg se Figur 2.2. Där kan den röda kurvan tolkas som en cosinusvåg, medan den blåa kurvan är en sinusvåg. En subcarrier är en högfrekvent sinus- eller cosinusvåg som används för att bära information, till exempel genom att ändra dess amplitud, frekvens eller fas. I QAM ändras både amplitud och fas för att badda in data i signalen. På så sätt kan fler bitar överföras per symbol, vilket ökar spektrumeffektiviteten. En symbol i QAM är en unik kombination av amplitud och fas, som representerar flera bitar data. Till skillnad från enklare modulationsmetoder, såsom BPSK där endast en bit överförs per symbol, kan QAM överföra flera bitar per symbol. Antalet bitar per symbol beror på vilken QAM-variant som används, exempelvis 4 bitar för 16-QAM eller 8 bitar för 256-QAM [12].



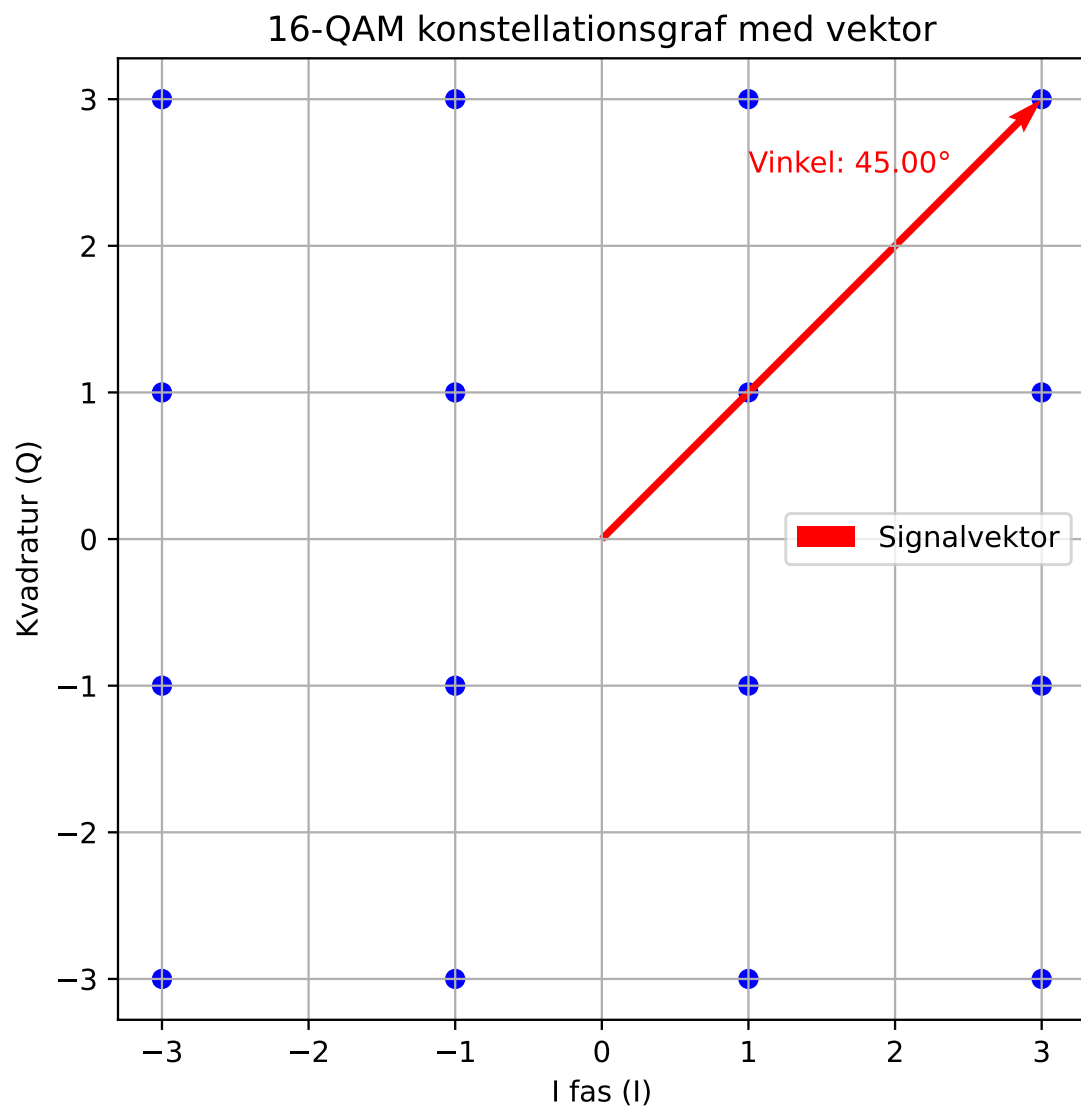
Figur 2.2: Exempel på IQ modulation [2]

2.2.1 Olika typer av QAM

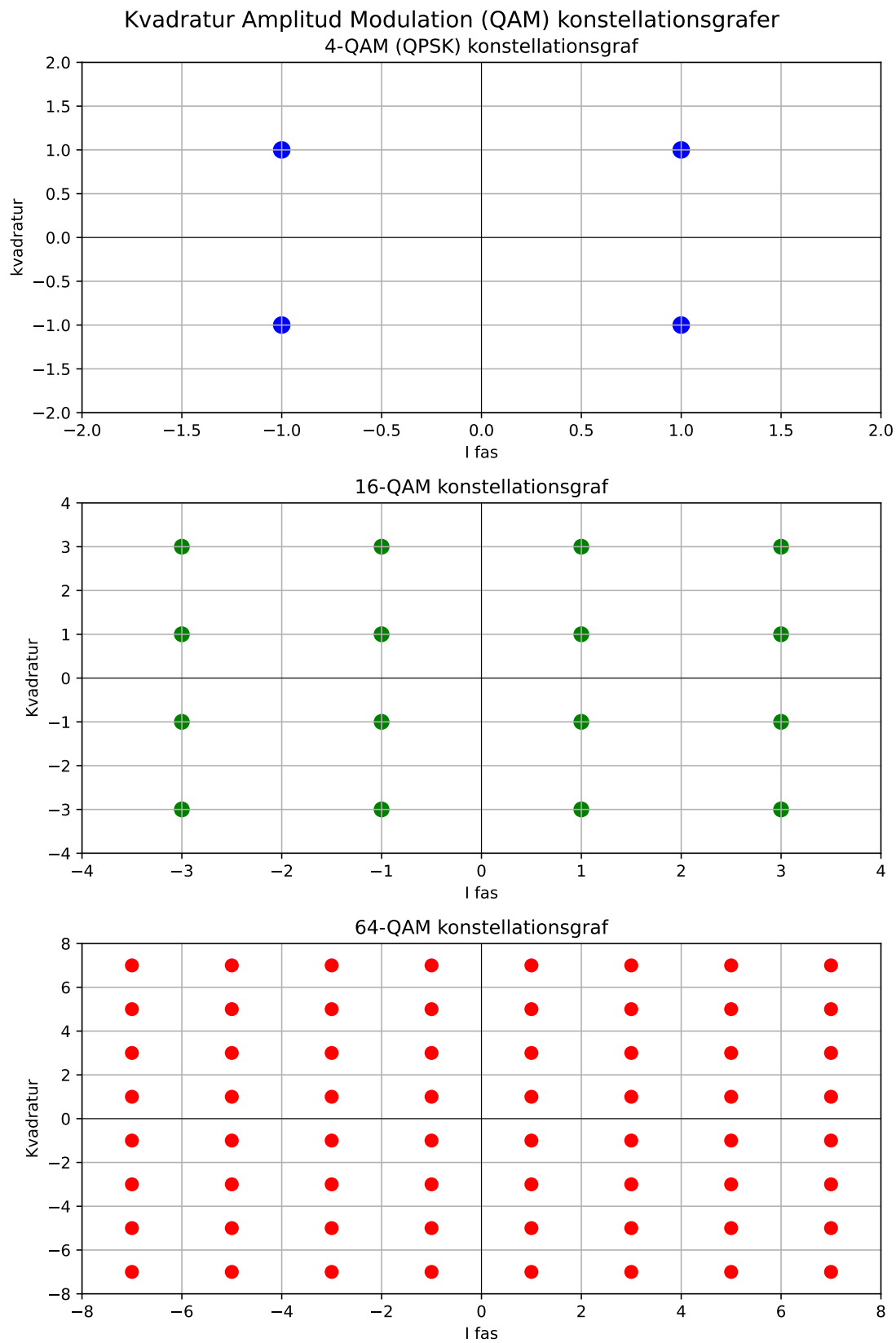
QAM kan ha olika typer, var och en med ett varierande antal punkter i ett så kallat konstellationsdiagram, vilket representerar olika nivåer av dataöverföring. Vid digital signalmodulering används ett konstellationsdiagram för att visa en graf över en QAM-signal. Detta diagram används för att analysera och felsöka kommunikationssystem. Diagrammet skiljer sig från In phase Quadrature (IQ)-modulering eftersom det visar hur både signalens styrka (amplitud) och dess fas ändras. Figur 2.3 visar ett exempel på ett konstellationsdiagram med typen 16-QAM. Varje punkt på diagrammet står för en symbol. I- och Q-axlarna visar hur signalens styrka förändras. Ju längre punkten är från origo(0,0), desto starkare är signalen. Punktens vinkel visar faser för signalen [3]. Varje symbol i konstellationsdiagramet representerar upp till 4 binära bitar (nollor och ettor) i ett 16-QAM-schema. Eftersom 4 bitar kan kombineras på $2^4 = 16$ olika sätt, finns det 16 möjliga symboler. Varje symbol motsvarar därmed ett unikt värde från 0000 till 1111

[13]. Den röda vektorn, som är en pil från origo till en av symbolpunkterna illustrerar en specifik signal som överförs i det digitala kommunikationssystemet. Vektorns riktning visar fasen (i detta fall 45°), och dess längd motsvarar signalens amplitud. På så sätt får man en tydlig visuell förståelse för hur symbolen representerar både fas och styrka i signalen.

De vanligaste varianterna av QAM är 4QAM, 16QAM, 32QAM, 64QAM och 256QAM. 4QAM, som också kallas Quadrature Phase Shift Keying (QPSK), innehåller fyra punkter i konstellationsdiagrammet, vilket gör att den kan sända 2 bitar per symbol [14]. Den används ofta i system där stabil dataöverföring är viktigare än hög kapacitet, särskilt i miljöer med mycket brus. 16QAM har 16 punkter i konstellationsdiagrammet, vilket innebär att den kan överföra 4 bitar per symbol, se Figur 2.4. Den används i digital-TV och kabelmodem eftersom den erbjuder en bra balans mellan hög dataöverföring och signalkvalitet. 64QAM har 64 punkter i konstellationsdiagrammet och kan sända 6 bitar per symbol. Den används främst i bredbandskommunikation där högre datahastigheter krävs. 256QAM har 256 punkter i konstellationsdiagrammet, vilket möjliggör överföring av 8 bitar per symbol. Den används ofta i höghastighetskabelmodem och digitala TV-sändningar, där både höga datahastigheter och pålitlig överföring är avgörande [14]. I det här arbetet används 256QAM.



Figur 2.3: QAM konstellationsgraf[3]



Figur 2.4: Fler eksempel på konstellationsgrafer [4]

2.3 BFP algoritm

Komprimeringen av data med BFP sker genom att kalkylera vad den gemensamma exponenten för ett Physical Resource Block (PRB) ska bli. Ett PRB har 12 rader av I och Q-data som används till QAM. I detta fall är I och Q-data en 16-bitars float mantissa, för enkelhetens skull kommer mantissans bitar hanteras som int. BFP algoritmen identifierar den mest signifikanta biten i talet och subtraherar med antalet bitar de slutgiltiga talen ska bli. Slutligen reduceras alla talen med exponenten. Se pseudokoden nedan i Kodlistning 2.1. Enligt pseudokoden fungerar BFP-algoritmen så här:

1. **Hitta maximi- och minimivärdena:** Algoritmen börjar med att identifiera maximi- ($\max V$) och minimi- ($\min V$) värdena bland alla reella och imaginära delar av de komplexa indatavärdena i ett PRB-block, betecknat som f_{PRB} .
2. **Beräkna det maximala absolutvärdet:** Därefter bestäms det maximala absolutvärdet. Detta säkerställer att både positiva och negativa provvärden kan representeras korrekt. Att subtrahera 1 från det negativa värdet förhindrar att bitrepresentationen överflödar vid konvertering till tvåkomplementformat.
3. **Beräkna den råa exponenten:** Den råa exponenten motsvarar positionen för den mest signifikanta biten i det största (absoluta) värdet inom PRB-blocket. Den beräknas genom att ta tvålogaritmen (bas 2) av $\max Value$, och används därefter för att bestämma den gemensamma exponenten.
4. **Bestäm exponenten för skalning:** Exponenten som används för skalning beräknas genom att subtrahera den önskade bitbredden ($iqWidth$, t.ex. 8 eller 9 bitar) från den råa exponenten. I detta arbete är $iqWidth = 9$. Detta steg avgör hur mycket skalning som krävs för att alla värden ska rymmas inom det angivna antalet bitar.

5. **Beräkna skalningsfaktorn:** Den faktiska skalningsfaktorn beräknas enligt:

$$\text{scaleFactor} = 2^{\text{iqWidth}-1-\text{rawExp}}$$

6. **Skala och kvantisera varje värde i blocket:** Varje reell och imaginär del av indatablocket multipliceras med skalningsfaktorn och kvantiseras (avrundas) för att minska dess bitbredd. Detta resulterar i de slutgiltiga komprimerade utgångsvärdena, cPRB.

```

1
2 // Find max and min maxV = max(Re(fPRB), Im(fPRB)), minV = min(Re(fPRB), Im(fPRB))
3 // Determine max absolute value
4 maxValue = max(maxV, |minV|-1)
5 /* msb of negative value can be one higher */
6
7 // Calculate exponent
8 rawExp = [floor(log2(maxValue) +1)] /* msb of maxV */
9 // Calculate shift value and limit to positive
10 exponent = max(rawExp - iqWidth + 1, 0)
11 // Determine right shift value
12 scaler = 2^(-exponent)
13 For iRe = 1:length(fPRB)
14 // Scale and round, mult. may be bit-shift, Quantize may be or-round
15     Re(cPRB(iRe)) = Quantize (scaler & Re(fPRB(iRe)))
16     Im(cPRB(iRe)) = Quantize (scaler & Im(fPRB(iRe)))
17 End

```

Listning 2.1: Psuedokod för PFB-Komprimering

Mottagen data ska även dekomprimeras och är exekveringsmässigt lättare då den gemensamma exponenten redan är funnen och inte behöver kalkyleras igen, se pseudokod nedan i Kodlistning 2.2.

```

1 scaler = 2^(exponent)
2 For iRe = 1:length(cPRB)
3 //Scale, this may be replaced with a bit-shift operation
4     Re(fPRB(iRE)) = scaler & Re(cPRB(iRE))
5     Im(fPRB(iRE)) = scaler & Im(cPRB(iRE))

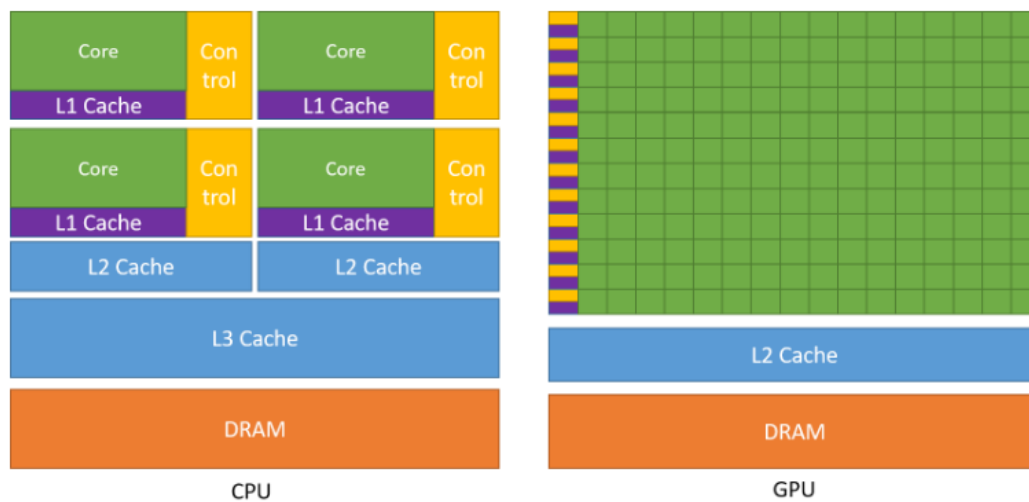
```

Listning 2.2: Psuedokod för PFB-Avkodning

Det finns andra algoritmer för komprimering än BFP och du kan läsa om dem i avsnitt 2.6 [6].

2.3.1 GPU vs CPU

Anledningen till att använda en GPU istället för en CPU är att GPU har en högre kapacitet för parallell exekvering. Det vill säga att bearbeta data eller utföra många uppgifter samtidigt jämfört med en CPU. Som nämnts i Avsnitt 1.3, med det förklarande exemplet med ägg: att koka tre ägg i en kastrull samtidigt tar mindre tid än att koka ett ägg i taget. Den största skillnaden mellan GPU och CPU är hur de är designade för att utföra olika uppdrag. CPU används ofta för att köra operationer i sekvens via färre trådar för snabbare exekvering. En GPU är designad för att kunna exekvera tusentals trådar parallellt. Genom att köra många trådar samtidigt kompenserar den för sin lägre prestanda vid exekvering av enskilda trådar, och uppnår därmed en avsevärt högre total prestanda (genomströmning) över alla trådar. GPU är specialiserad för många parallella beräkningar och har en specifik arkitektur vilket gör att fler transistorer ägnas åt databehandling snarare än datacache och flödeskontroll. Figur 2.5 visar ett exempel på distribution av chip resurser för en CPU jämfört med en GPU[5]. Som syns i figuren, har CPU:n ett fåtal kraftfulla kärnor, var och en med sin egen kontrollogik och dedikerade L1-cache, tillsammans med delade L2- och L3-cacher. Den är designad för komplexa,



Figur 2.5: Illustration av skillnaden i processorarkitektur mellan CPU och GPU [5]

sekventiella uppgifter med fokus på prestanda med låg latens. Däremot innehåller GPU många enklare kärnor arrangerade i ett rutnät med minimal kontrollogik, som delar en stor L2-cache. Denna struktur är optimerad för massivt parallella uppgifter, som flyttalsberäkningar [5].

2.4 CUDA

CUDA är en programmeringsmodell som exekverar uppgifter parallellt [15]. Denna dataprocesseringsplattform utvecklades av NVIDIA för att möjliggöra komplexa beräkningar genom att använda GPU. Användningen av CUDA underlättar för utvecklare att skriva kod som delar upp tunga beräkningar i många små delar som kan köras samtidigt på tusentals GPU-kärnor. Detta gör det möjligt att kraftigt öka prestandan för beräkningsintensiva applikationer. I GPU-accelererade applikationer körs den sekventiella delen av arbetsbelastningen på CPU, som är optimerad för enkeltrådad prestanda, medan den beräkningsintensiva delen av applikationen körs på tusentals GPU-kärnor parallellt. CUDA programmeras ofta i populära språk som C och C++[15].

2.4.1 Anledningen till att använda CUDA

Syftet med CUDA är att kunna skapa massivt parallella applikationer. Det ger möjligheter för utvecklare att använda det kraftfulla programmeringsspråket C/C++ för att exekvera komplicerade algoritmer som accelereras av tusentals parallella trådar som körs på GPU. Många utvecklare använder CUDA för att implementera beräknings- och bandbreddskrävande applikationer, inklusive de bibliotek och ramverk som ligger till grund för det som kallas Deep Learning inom Artificial Intelligence (AI)[7].

2.4.2 Arbetsflödet i CUDA

För att GPU ska komma åt datan måste den presenteras i så kallad device memory. CUDA tillhandahåller Application Programming Interface (API)er för att allokera device memory och hantera dataöverföring mellan host (CPU) och device memory[16]. Följande är de vanliga stegen i CUDA-arbetsflödet:

1. Allokera host-minne och initialisera host-data.
2. Allokera device memory (se Avsnitt 2.4.4).
3. Överföra indata från CPU till GPU.
4. Exekvera kärnor.
5. Överföra utdata från GPU till CPU.
6. Avallokera device memory.

2.4.3 Att programmera med CUDA i C/C++:

När man programmerar med CUDA använder man tekniskt sett samma syntax som i C/C++, men det finns naturligtvis vissa skillnader. Exempel: anta att vi ska skriva

ett enkelt program för addition, där vi adderar två vektorer och lagrar summan i en tredje vektor. Det första man gör i detta fall är att skriva en additionsfunktion som ska kunna köras på GPU. En sådan funktion kallas en kernel i CUDA. För att indikera att funktionen ska köras på GPU markerar man den med nyckelordet *global*. Detta talar om för CUDA C++-kompilatorn att funktionen exekveras på GPU men kan anropas från CPU-koden. Kodlistning 2.3 och kodlistning 2.4 visar skillnaden mellan hur en vektoradditionsfunktion kan implementeras på en CPU respektive en GPU [7]. Som figurerna visar har båda funktionerna i princip samma syntax. Båda beräknar summan av två vektorer och sparar resultatet i en ny vektor *y*. Den enda skillnaden är att i Kodlistning 2.4 måste nyckelordet *global* anges i början av funktionen för att markera att det är en GPU-kärna (kernel).

```
1 // function to add the elements of two arrays
2 void add(int n, float *x, float *y){
3     for(int i = 0; i < n; i++)
4         y[i] = x[i] + y[i];
5 }
```

Listning 2.3: CPU exempel på seriell vektoraddition [7]

```
1 // CUDA kernel to add the elements of two arrays on the GPU
2 __global__ void add(int n, float *x, float *y)
3 {
4     for(int i = 0; i < n; i++)
5         y[i] = x[i] + y[i];
6 }
```

Listning 2.4: GPU exempel på seriell vektoraddition [7]

2.4.4 Minnesallokering i CUDA

Precis som man allokerar minne för data i C/C++ med *malloc()*, används *cudaMalloc()* för att allokera minne på GPU. Detta instruerar kompilatorn att allokera tillgängligt minne för beräkningar på GPU:n. För att frigöra det allokerade minnet anropas *cudaFree()* efter att programmet har exekverats. I Kodlistning 2.5 visas hur minneshantering sker vid allokering och frigöring av minne i CUDA[7].

```
1  int main(void)
2  {
3      int N = 1<<20;
4      float *x, *y;
5
6      // Allocate Unified Memory accessible from CPU or GPU
7      cudaMallocManaged(&x, N*sizeof(float));
8      cudaMallocManaged(&y, N*sizeof(float));
9      ...
10     cudaFree(x);
11     cudaFree(y);
12     return 0;
13 }
14
```

Listning 2.5: Minnesallokering i CUDA [7]

2.4.5 Antal block och trådar i CUDA

För att kunna exekvera additionsfunktionen som visats ovan, behöver den anropas från main-funktionen. Det vill säga, vi måste starta kernel-funktionen. Syntaxen för detta kan se ut så här enligt kodlistning 2.6.

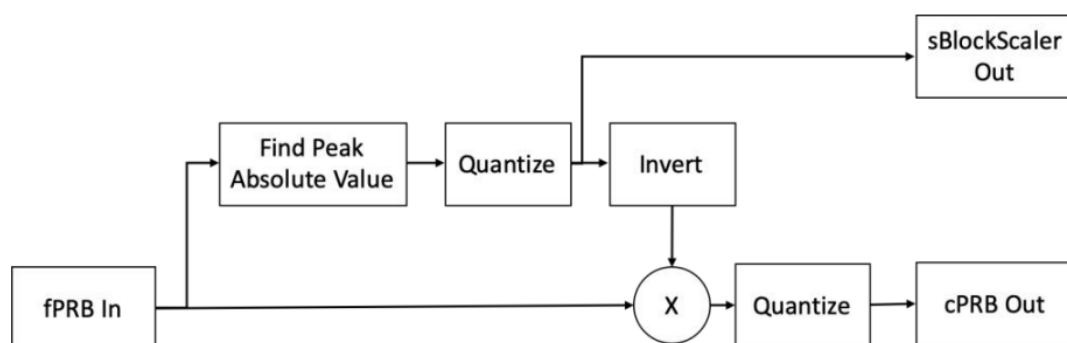
```
1
2  int main(void)
3  {
4      ...
5      add<<<1,256>>>>(N, x, y);
6      ...
7      return 0;
8  }
```

Listning 2.6: Exempel på Kernelanrop i CUDA [7]

Anropsfunktionen specificerar för CUDA hur många parallella trådar som ska tilldelas vid exekvering på GPU. Enligt figuren anges två parametrar. Den första parametern, som är 1 anger antalet block som ska köras i CUDA. Den andra parametern, som är 256 anger hur många trådar som ska tilldelas per CUDA-block. Detta ligger i en grid eller Streaming Multiprocessor (SM) som består av fyra mindre enheter, så kallade warps, där varje warp innehåller 32 trådar. Detta innebär att inte alla trådar kan exekveras samtidigt, vilket kan öka exekveringstiden [7].

2.5 Metoder

Programmet använder trådar för att parallellt bearbeta flera element, vilket möjliggör ett effektivare utnyttjande av grafikkortets resurser. I detta sammanhang används SM, vilket är en delmängd av de resurser som finns tillgängliga på GPU och ibland benämns som en streaming multiprocessor. För att kunna utnyttja en större andel av grafikkortets kapacitet är användningen av SM:er avgörande, särskilt vid beräkningstunga uppgifter. Det är i praktiken det enda sättet att aktivera mer än en bråkdel av de tillgängliga kärnorna på GPU.



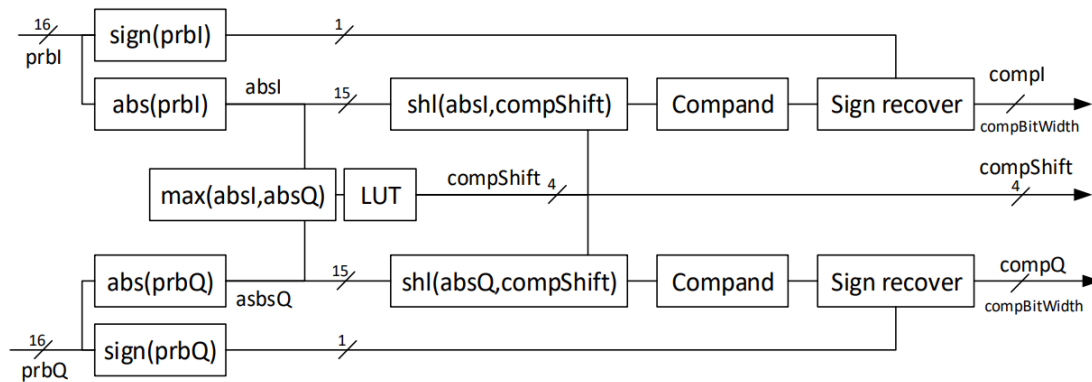
Figur 2.6: Diagram på Block scaling algoritm [6]

För att säkerställa korrekt exekveringsordning mellan trådar samt mellan värd och enhet används synkroniseringsmekanismerna `cudaDeviceSynchronize()` och `syncthreads()`. Dessa är nödvändiga för att garantera korrekt och deterministisk körning [5].

Under utvecklingen användes verktyget Nsight Systems för att analysera programrets beteende vid körning. Nsight Systems är ett verktyg från NVIDIA som ger utvecklaren en detaljerad översikt över programmets exekvering, inklusive alla anrop till CUDA-APIet. Verktyget visualiserar dessa anrop längs en tidslinje och visar både vilken stream de tillhör och vilken typ av operation de utför. Den information som presenteras av Nsight Systems används därefter för att vidare optimera programmet [9].

2.6 Liknande arbeten

Det finns algoritmer liknande BFP-komprimering som är värda att nämnas. Blockskalad komprimering och μ -law komprimering som kör på samma typ av data. I kapitel 2.4.3 finns även ett enklare exempel på multitrådning och parallell exekvering i CUDA

Figur 2.7: Diagram på μ -law algoritmen [6]

2.6.1 Block-skalad kompression

Block-skalad kompression representerar inte data i PRB blocket som en mantissa och exponent utan värden som är en serie av produkter och ett gemensamt skalvärde. Denna metod liknar BFP men den använder en fast skalningsfaktor för alla PRB inom samma överföring, medan BFP enbart har samma inom en PRB. Kompressionsprocessen vilar på att varje block skalas med en enda normaliseringsfaktor som är baserad på det maximala absoluta värdet i blocket. Den skalade datan kvantiseras och sänds tillsammans med skalningsfaktorn, vilket leder till lägre beräkningskomplexitet.[6]

2.6.2 μ - Law kompression

μ -law-kompression är en kombination av bitförskjutning och en uppskattning som görs på resultatet. Signbit (som är den bit som talar om ifall det ursprungliga värdet är positivt eller negativt) och data hanteras separat. Algoritmen är baserad på en icke-linjär kvantiseringssteknik som syftar till att minska antalet bitar som behövs vid överföring genom att effektivt koda IQ-data, samtidigt som signalkvaliteten bevaras. Små amplitudvärden kvantiseras med högre noggrannhet, medan större värden får lägre noggrannhet. [6]

2.6.3 Vektoraddition projekt med parallellisering

I Kodlistning 2.7 visas ett CUDA program som utför en vektoraddition parallellt. Detta program adderar två stora vektorer A och B. Beräkningen utförs parallellt genom att använda flera trådar på GPU. Funktionen *global_add_vectors* är en CUDA kernel som körs på GPU och anropas från CPU. För att varje tråd i CUDA tilldelas en unik trådid, används formeln

```
int id = blockDim.x * blockIdx.x + threadIdx.x;
```

Där

- *blockIdx.x* : block index inuti grid.
- *threadIdx.x* : Tråd index inuti blocket
- *blockDim.x* : totala antal trådar i dimensionen x inuti blocket

if(id < N)c[id] = a[id] + b[id]; Den raden utför vektoradditionen parallellt om tråden inte kommer åt minnet som är utanför intervallet.

2.6.4 Kernel-exekvering (starta CUDA kärnor)

$$\text{int blk_in_grid} = \left\lceil \frac{\text{float}(N)}{\text{thr_per_blk}} \right\rceil$$

Antal trådar per block är 256 och antal block i grid = $\frac{1048576}{256} = 4096$. Detta innebär att 4096 block startade, där var och ett har 256 trådar, vilket resulterar i 1,048,576 trådar parallellt. Slutligen startas kernel genom att anropa funktionen *add_vectors* `add_vectors(<<< blk_in_grid, thr_per_blk >>>)(d_A, d_B, d_C);` med 4096 block och 256 trådar per block [7].

1

2 `#include <stdio.h>`

```
3
4 // Size of array
5 #define N 1048576
6
7 // Kernel
8 __global__ void add_vectors(double *a, double *b, double *c)
9 {
10     int id = blockDim.x * blockIdx.x + threadIdx.x;
11     if(id < N) c[id] = a[id] + b[id];
12 }
13
14 // Main program
15 int main()
16 {
17     // Number of bytes to allocate for N doubles
18     size_t bytes = N*sizeof(double);
19
20     // Allocate memory for arrays A, B, and C on host
21     double *A = (double*)malloc(bytes);
22     double *B = (double*)malloc(bytes);
23     double *C = (double*)malloc(bytes);
24
25     // Allocate memory for arrays d_A, d_B, and d_C on device
26     double *d_A, *d_B, *d_C;
27     cudaMalloc(&d_A, bytes);
28     cudaMalloc(&d_B, bytes);
29     cudaMalloc(&d_C, bytes);
30
31     // Fill host arrays A and B
32     for(int i=0; i<N; i++)
33     {
34         A[i] = 1.0;
35         B[i] = 2.0;
```

```
36  }
37
38  // Copy data from host arrays A and B to device arrays d_A and d_B
39  cudaMemcpy(d_A, A, bytes, cudaMemcpyHostToDevice);
40  cudaMemcpy(d_B, B, bytes, cudaMemcpyHostToDevice);
41
42  // Set execution configuration parameters
43  //thr_per_blk: number of CUDA threads per grid block
44  //blk_in_grid: number of blocks in grid
45  int thr_per_blk = 256;
46  int blk_in_grid = ceil( float(N) / thr_per_blk );
47
48  // Launch kernel
49  add_vectors<<< blk_in_grid, thr_per_blk >>>(d_A, d_B, d_C);
50
51  // Copy data from device array d_C to host array C
52  cudaMemcpy(C, d_C, bytes, cudaMemcpyDeviceToHost);
53
54  // Verify results
55  double tolerance = 1.0e-14;
56  for(int i=0; i<N; i++)
57  {
58    if( fabs(C[i] - 3.0) > tolerance)
59    {
60      printf("\nError: value of C[%d] = %d instead of 3.0\n\n", i, C[i]);
61      exit(1);
62    }
63  }
64
65  // Free CPU memory
66  free(A);
67  free(B);
68  free(C);
```

```
69
70 // Free GPU memory
71 cudaFree(d_A);
72 cudaFree(d_B);
73 cudaFree(d_C);
74
75 printf("\n-----\n");
76 printf("__SUCCESS__\n");
77 printf("-----\n");
78 printf("N                = %d\n", N);
79 printf("Threads Per Block = %d\n", thr_per_blk);
80 printf("Blocks In Grid    = %d\n", blk_in_grid);
81 printf("-----\n\n");
82
83 return 0;
84 }
85
```

Listning 2.7: CUDA kod exempel på vektoraddition med trådar

2.7 Optimering

I CUDA tillhandahålls flera varianter av funktioner som används för att optimera arbetslasten på GPU. De funktioner som används i detta projekt är `cudaMallocHost()`, `cudaLaunchKernel()` och `cudaMemoryAsync()`.

Funktionen `cudaMallocHost()` liknar `cudaMalloc()`, men den förstnämnda möjliggör snabbare dataöverföring från host till device-minne. Detta uppnås genom att minnesallokeringen sker direkt i det s.k. pinned vädminnet, vilket undviker att minnet fragmenteras eller hamnar i långsammare sekundärminne [17].

Eftersom GPU kompilerar koden vid första exekveringstillfället, medför detta en fördröjning för det initiala blocket i en iteration. `cudaLaunchKernel()` möjliggör dock

att kompileringen sker innan själva körningen påbörjas, vilket gör att den första tidmätningen inte påverkas av den inledande kompileringen på grafikkortet [17].

GPU består av flera SM, vilka i sin tur hanterar så kallade warps grupper av trådar som exekveras parallellt. Normalt tilldelas arbetet ett standard-SM, men för att maximera graden av parallellism bör så många warps och SM som möjligt utnyttjas.

`cudaMemoryAsync()`, i kombination med en alternativ metod för programuppstart som inkluderar ett specifikt argument, möjliggör körning i en angiven stream. Detta möjliggör överlappning av dataöverföring och exekvering, vilket kan förbättra prestandan vid komplexa arbetsflöden [17].

Kapitel 3

Design

Inledningsvis introduceras och definieras problemställningen både på en övergripande och detaljerad nivå. Vi beskriver hur BFP-algoritmen fungerar i det nuvarande systemet, samt vilken typ av data som hanteras vid exekveringen av algoritmen. Därefter presenteras vår föreslagna lösning på problemet. Här förklarar vi hur vi använder parallell exekvering med hjälp av CUDA för att effektivisera beräkningarna, samt redogör för vårt tillvägagångssätt vid implementationen. Följt av designaspekten på lösningen. Vi beskriver hur programvaran planerades och strukturerades, baserat på den tekniska information och de krav som tillhandahölls av vår uppdragsgivare. Slutligen behandlas verifiering av lösningen. Här diskuteras vilka metoder och faktorer som används för att säkerställa att implementationen är korrekt, samt vilka aspekter som kan påverka noggrannhet och prestanda i det slutgiltiga resultatet.

3.1 Problembeskrivning

I det nuvarande systemet körs BFP-algoritmen sekventiellt på en kärna i en CPU. Generellt sett har en kärna i en CPU bättre prestanda än en kärna i en GPU. Men eftersom en GPU har fler kärnor än en CPU, kan man utnyttja denna kapacitet hos GPU för att köra

komplexa algoritmer parallellt och därmed uppnå kortare exekveringstider. Problemet handlar om att implementera BFP-algoritmen på en GPU för att utvärdera om det är värt att investera i en sådan lösning för basstationen. Vi skulle skapa olika varianter av CUDA program för att köra BFP-algoritmen och undersöka om vi skulle kunna uppnå kortare exekveringstider jämfört med de tider som erhålls i det nuvarande systemet.

3.1.1 Hur nuvarande system fungerar

Nuvarande system kör BFP-komprimering och dekomprimering seriellt och exekveras på CPU. Algoritmen är något simpel men ska utföras på mycket data. Varje PRB block innehåller 12 rader av I och Q-data som är 16 bit eller 2 Bytes vardera. Detta resulterar i $12 \times 2 \times 2$, vilket är totalt 48 Bytes. Det här ska utföras på upp till 2184 block parallellt vilket är cirka 105kB på midband och 400 block på lågband vilket är 19.2 kBytes [6].

3.1.2 Lågband och midband

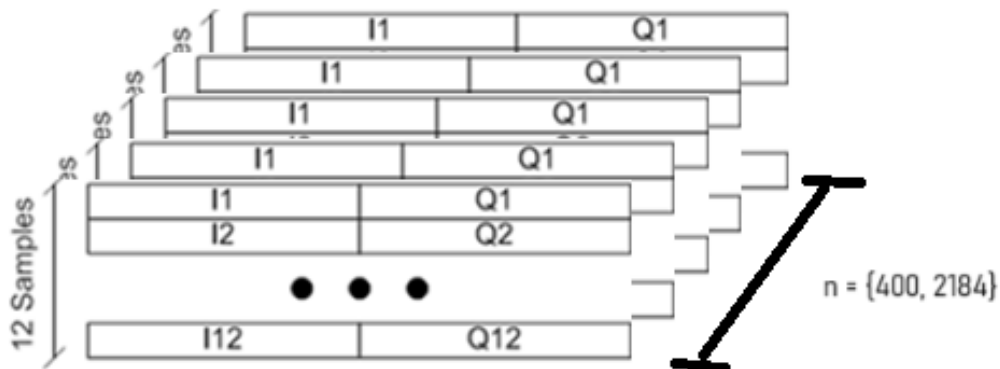
Enligt den information vi fått från TietoEvry hänvisar midband och lågband inom O-RAN till olika frekvensområden som används för trådlös kommunikation. I praktiken körs BFP-algoritmen på en basstation, där data komprimeras och därefter skickas vidare till en cell. En cell representerar ett geografiskt område som en radiomast täcker. På varje radiomast finns ett antal antenner som hanterar kommunikationen inom sin cell. Lågband har en bandbredd på 20 MHz och använder 4 antenner. Den har kapaciteten vid denna frekvens att överföra 100 PRB per antenn, vilket innebär att en basstation för en cell ska överföra 400 PRB på 71.4 mikrosekunder till antennerna. Midband har en bandbredd på 100 MHz och använder 8 antenner. Den har kapaciteten att överföra 273 PRB per antenn, vilket innebär att en cell ska överföra 2184 PRB på 35.7 mikrosekunder till antennerna. Lågband och midband har olika tidskrav för att överföra en slot. En tidsslot för lågband är 1 ms, medan den är 0,5 ms för midband. Varje tidsslot består

av 14 symboler, där varje symbol omfattar 400 PRB för lågband och 2184 PRB för midband. Detta ger följande tider:

$$\left(\frac{1 \text{ ms}}{14 \text{ symbols}} \right) = 71.4 \mu\text{s per symbol för lågband}$$

$$\left(\frac{0.5 \text{ ms}}{14 \text{ symbols}} \right) = 35.7 \mu\text{s per symbol för midband}$$

Målet var att utveckla ett program som klarar att bearbeta data inom de angivna tidsgränserna för lågband och midband. I Figur 3.1 representeras den bearbetade datan som en tredimensionell struktur. Varje PRB-block består av 12 rader med I- och Q-data samt 2 kolumner, där den första kolumnen representerar I-data och den andra kolumnen representerar Q-data. Variabeln n i figuren visar den tredje dimensionen, som representerar antalet PRB-block: 400 för lågband och 2184 för midband.



Figur 3.1: Data för I och Q i ett PRB block [6]

3.2 Strategier för att minska exekveringstider

För att kunna lösa problemet måste vi skapa ett CUDA-program som utnyttjar så många GPU-kärnor som möjligt för att kunna utföra arbetet parallellt i största möjliga utsträck-

ning. Teoretiskt sett bör BFP-algoritmen exekveras snabbare ju fler kärnor som tilldelas arbete. Därför planerade vi att skapa flera olika program, där varje program använder ett bestämt antal kärnor, för att undersöka vilket av dessa ger bäst resultat. Syftet var att verifiera om det stämmer att det program som tilldelas flest kärnor också ger bäst prestanda, i enlighet med teorin.

3.2.1 Planen för att lösa problemet

För att lösa problemet planeras projektet att genomföras i flera iterationer. Avsikten är att i varje iteration utveckla en förbättrad version av parallell exekvering, med målet att undersöka olika lösningar och slutligen ta fram en version som ger bästa möjliga resultat. I den första iterationen planeras utvecklingen av ett enkelt program i C som körs på CPU. Detta program ska fungera som referens för de varianter som senare ska köras parallellt på GPU. De efterföljande iterationerna planeras att bestå av olika varianter av CUDA-program, där varje iteration fokuserar på att implementera och utvärdera olika parallelliseringstekniker för att exekvera BFP-algoritmen. Mer detaljer om de planerade parallelliseringsteknikerna finns under avsnittet "Implementation" i kapitel 4. Under planeringsfasen analyseras även möjligheterna att parallellisera ytterligare delar av arbetet med hjälp av CUDA. En del av denna plan är att använda så kallade CUDA-streams i samtliga kommande CUDA-program. CUDA-streams möjliggör samtidig exekvering och överlappning av operationer, såsom kärnexekvering och minnesöverföringar, vilket kan förbättra prestandan avsevärt jämfört med ett traditionellt CUDA-program som använder en enda (standard) stream, där alla operationer (minneskopior, kärnor) serialiseras, vilket förhindrar överlappning och minskar prestandan. Genom att utveckla och testa de olika varianterna av CUDA-program enligt ovanstående plan, samt genom att använda CUDA-streams i dessa program, är målet att ta fram en slutlig version anpassad för lågband respektive midband. Dessa versioner ska uppfylla de krav som ställs på exekveringstider och datavolymer inom givna tidsramar. I Tabell 3.1 presenteras sex

Komponent	Algoritm	Krav	Variant	överföring	Iteration
CPU	Komprimering	Midband	Seriell	FALSE	1
CPU	Dekomprimering	Midband	Seriell	FALSE	1
GPU	Komprimering	Midband	Seriell	FALSE	2
GPU	Dekomprimering	Midband	Seriell	FALSE	2
GPU	Komprimering	Midband	Parallell i PRB	FALSE	3
GPU	Komprimering	Midband	Parallell i PRB	FALSE	3
GPU	Komprimering	Midband	Alla PRB parallellt	TRUE	4
GPU	Dekomprimering	Midband	Alla PRB parallellt	TRUE	4
GPU	Komprimering	Midband	Full Parallell	TRUE	5
GPU	Dekomprimering	Lågband	Full Parallell	TRUE	5

Tabell 3.1: Tabell över de mjukvaror vi utvecklade under arbetet och diskuterar i rapporten

kolumner. Den första kolumnen, *Komponent*, specificerar vilken hårdvara algoritmen ska exekveras på. Den andra kolumnen, *Algoritm*, anger vilken algoritm som används. Den tredje kolumnen visar vilket krav programmet är utvecklat för. Kolumnen *Variant* beskriver vilken specifik förbättring som programmet implementerar. Den femte kolumnen specificerar om programmet beaktar överföringstiden på PCIe. Den sista kolumnen visar vilken iteration programmet tillhör.

3.3 Lösningen på designnivå

Den slutliga designen vi tog fram för att köra programmet beskrivs i Figur 3.2. Ett hjälpprogram som är skrivet i C-språket, används för att generera data som komprimeras. Programmet anropar funktionen *rand()*, som används för att slumpmässigt generera data av typen 32-bitars heltal (integer). Den genererade datan skrivs sedan till en separat fil. I steg 1 läses data in från den separata filen. Filen innehåller 218400 rader, där varje rad innehåller data för ett block bestående av 12 heltal. I steg 2 delas datan upp i två lika stora delar genom att bit-skifta varje heltal 16 bitar och tilldela dem varsin 16-bitars short-variabel för att representera I- och Q-data i PRB-blocket. Dessa I- och Q-

värden sparas i en specifik datastruktur som allokeras i primärminnet, och som inte kan flyttas till sekundärminnet tack vare användningen av funktionen *cudaMallocHost()*. Strukturen består av en lista med 24 heltal som representerar ett PRB, där varje heltal är två byte stort, samt ett heltal som representerar exponenten. I steg 3 skickas PRB-blocket till GPU för att köra BFP-algoritmen, som används för att komprimera och dekomprimera datan. I steg 4, när komprimering och dekomprimering är klara, skickas datan tillbaka till CPU. I steg 5, när programmet har exekverats färdigt, avallokeras alla dynamiska listor både i värdenheten och GPU-enheten. Slutligen skrivs alla tider för algoritmens exekvering ut, i mikrosekunder, för att visa den totala bearbetningstiden.

3.4 Verifiering av lösningen

För att verifiera att implementationen är korrekt planeras att demonstrera att exekveringstiderna kan hållas under de angivna tidsgränserna för lågband och midband. En viktig aspekt att beakta under projektets gång är de faktorer som kan påverka resultatet, såsom den hårdvara som används. Baserat på de två hårdvaruplattformar som finns tillgängliga planeras en jämförelse av deras prestanda, i syfte att utvärdera vilken som är mest lämplig för detta arbete. Det planeras att undersöka samt att använda verktyg för felsökning och analys för att identifiera och hantera eventuella buggar eller defekter som kan uppstå under utvecklingen. Det är för att säkerställa att algoritmen är korrekt implementerad. Detta planeras att kontrolleras genom att jämföra indata och utdata, och verifiera att ingen signifikant informationsförlust sker vid komprimering och dekomprimering. Dessa faktorer och deras potentiella påverkan på resultaten behandlas närmare i följande kapitel.

3.4.1 Hårdvara

Följande är en lista över den hårdvara som var installerad på servern där algoritmen kördes och programmet utvecklades.

Komponent	Specifikationer
CPU	Intel Core i9-11900, 16-core, 32-threads, 2.5/5.2 GHz
RAM	DDR4 3200 MHz, 64 GB
SSD	2 TB M.2 2280, PCIe Gen3 x4, 4700 MB/s läs, 4200 MB/s skriv
GPU	NVIDIA GeForce RTX 3090, PCIe 4.0, 24 GB GDDR6X, Ampere, 10496 CUDA-kärnor

Tabell 3.2: Hårdvaruspecifikationer för gamla utvecklingsservern

Ungefär halvvägs in i arbetet installerades en ny server på företaget. Företaget Tieto ville då att vi skulle jämföra tider mellan dessa servrar för att bättre ge en insikt i vilken hårdvara som var mest rimlig för en implementering av en bas-station.

Komponent	Specifikationer
CPU	Intel Core i9-14900KF, 24-core, 32-threads, upp till 6.0 GHz
RAM	DDR5 6000 MHz, 64 GB
SSD	2 TB M.2 NVMe, PCIe 4.0 x4, 7300 MB/s läs, 7000 MB/s skriv
GPU	NVIDIA GeForce RTX 5090, PCIe 5.0, 32 GB GDDR7, 21760 CUDA-kärnor

Tabell 3.3: Uppdaterade hårdvaruspecifikationer

Med tanke på att den nya servern använder ett RTX 5090-grafikkort, skulle vi i så fall kunna få bättre exekveringstider, eftersom 5090 har en högre basfrekvens för kärnorna 2,01GHz jämfört med 3090, som endast har 1,395GHz.

3.4.2 Mjukvara

Under projektets gång användes olika mjukvaror för att genomföra arbetet. Dessa valdes eftersom arbetsgivaren gav oss tillgång till en servermiljö och önskade att mätningarna skulle utföras där.

Vi arbetade i en servermiljö med en specialdistribution av Ubuntu, kallad TUbuntu. Utvecklingsmiljön för mjukvaran var Visual Studio Code (VS Code). Versionshantering sköttes med hjälp av Git, vilket möjliggjorde spårning av ändringar i källkoden under utvecklingens gång.

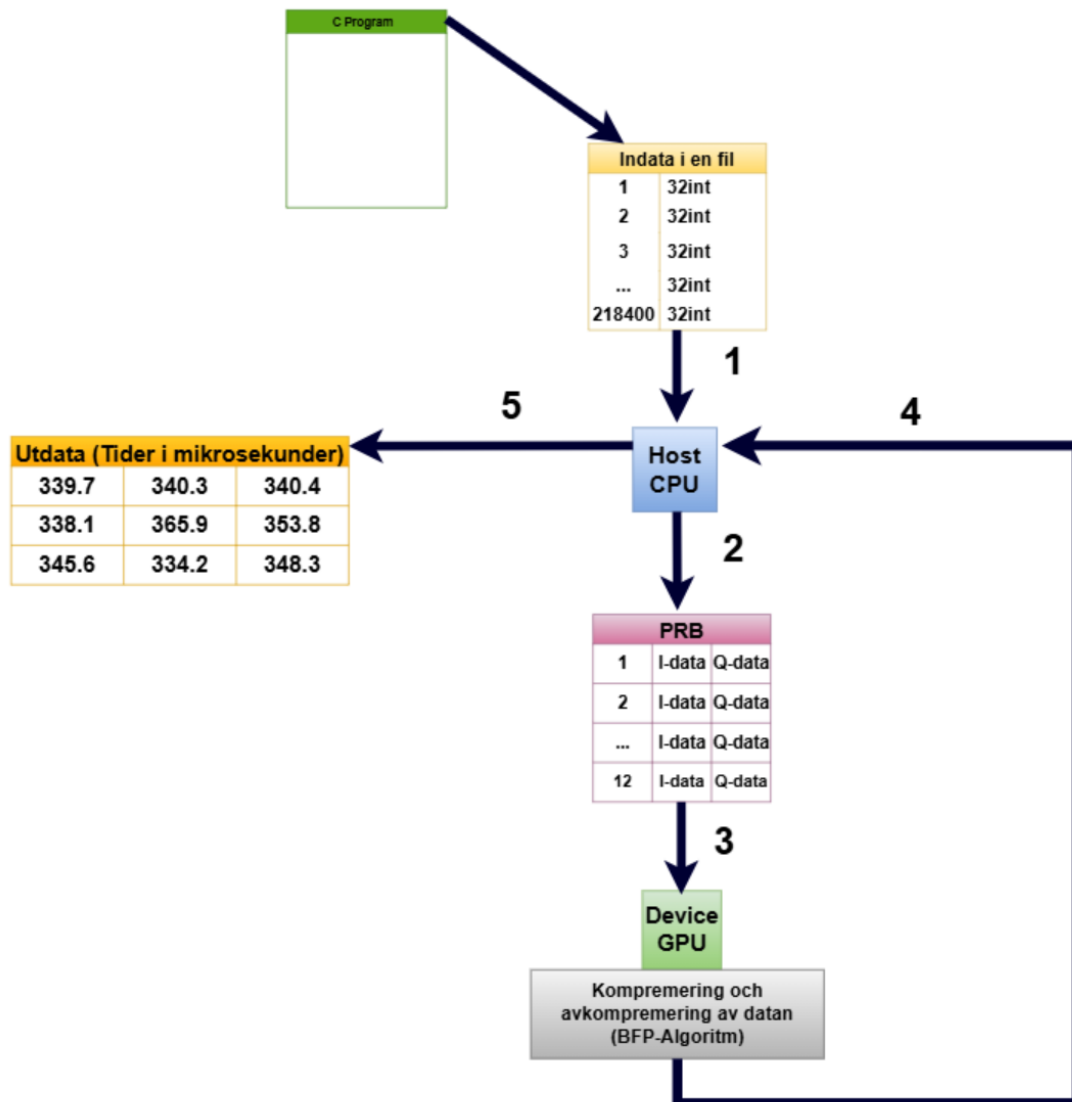
Fjärråtkomst till servern skedde via programmen NoMachine[18] och MobaXterm[19], vilka används för att låta en användare styra en dator på distans över nätverk eller internet¹.

För analyseringen av programmet användes verktygen NVIDIA Nsight Systems. Detta användes i slutskedet av utvecklingen för att analysera programmets prestanda och identifiera möjligheter till optimering och förbättring.

3.4.3 Parametrar

Programmet har ett antal parametrar. Som beskrevs i Avsnitt 3.3 tar programmet emot en fil med data som innehåller heltal specifikt 218400 rader med 12 heltal per rad och genererar som utdata tider i nanosekunder. Utöver detta har programmet parametrar för att ange om det ska köras för lågband eller midband, samt parametrar för antalet SM och antalet celler. Detta möjliggör att programmet kan bearbeta större datamängder och använda fler processorkärnor i samma körning.

¹Mer information om fjärråtkomstprogrammen finns via de angivna länkarna. TUbuntu är den Linux-distribution som användes på servrarna och motsvarar i stort sett en vanlig Ubuntu-installation.



Figur 3.2: Programmets design och exekvering process

Kapitel 4

Implementation

I det här kapitlet beskrivs den mjukvara som utvecklades för att lösa uppgiften.

Exakt som i Tabell 3.1 kommer dem programmen demonstreras: **Iteration 1 - C-variant:** Detta program körs sekventiellt på en CPU-kärna och används som referens för att jämföra skillnaden mellan att exekvera algoritmen på en CPU och en GPU.

CUDA-program: Här skapade vi olika CUDA-program som använde olika parallelliseringstekniker för att köra algoritmen. Syftet var att undersöka vilken teknik som gav bäst resultat. Programvarianterna är följande:

Iteration 2 - Exekvering på en kärna på GPU: Detta program exekveras på en enda kärna, precis som i C-varianten. Syftet är att visa att arbete på en kärna i CPU ofta går snabbare än på GPU, vilket betonar vikten av att utnyttja GPU:ns kapacitet för parallell exekvering.

Iteration 3 - Parallell exekvering internt i ett block: Exekveringen sker parallellt för alla element inom ett PRB, men 2184 PRB behandlas sekventiellt.

Iteration 4 - Parallell exekvering över alla block: I denna variant körs blocken parallellt, men varje PRB innehåller endast en tråd. Detta möjliggör parallell exekvering av blocken, medan arbetet inom varje block sker seriellt.

Iteration 5 - Fullständig parallellisering (en tråd per element): Varje I- och Q-

element tilldelas en egen tråd, vilket gör att varje kärna kan arbeta på ett enskilt element.

4.1 Implementation

Utvärderingen av GPU för att utföra PRB algoritmer skedde i flera iterationer av program. De utökades för att testa högre hastighet på utförandet och optimera med olika funktioner och parallell exekvering. I avsnitt 4.2 utvärderades ett seriellt C-program på servern för referens. I avsnitt 4.3 utvecklades och optimerades olika CUDA program. För att ha bra tider teoretiskt sett skulle programmet utnyttja så många kärnor som möjligt och utföra allt som går parallellt. Inledningsvis när programmen utvecklades exekverade vi enbart för 100 PRB men för resultatet ändrade vi till 2184 för att få tider som reflekterade en symbol på midband.

4.2 C variant- Referensprogram

Den första implementeringen av BFP körde algoritmen seriellt på CPU (se Kodlistning 4.1). Vi skapade en datatyp för blocket som innehöll listor för datavärden samt fält för max-, min- och den delade exponenten. För att öka tydligheten implementerades dessutom hjälpfunktioner såsom max, min och listversioner av dessa. Observera att komprimeringsalgoritmen använder for-loopar för att hantera listor, till skillnad från kommande varianter i följande kapitel som inte gör det. Programmet tog in data från en separat fil. Denna data bearbetades på CPU och lades i en lista. Exekveringstider för algoritmerna mättes för att jämföra med de senare iterationerna.

```
1  #include <stdio.h>
2  #include <math.h>
3  #include <stdlib.h>
4  #include <time.h>
```

```
5  #define NUMBER_OF_BLOCKS 2184
6  #define BLOCKSIZE 12
7  #define IQWIDTH 9
8  #define ITERATIONS 10000
9  #define DEBUG_DATA 0
10
11 typedef struct{
12     short Im[BLOCKSIZE];
13     short Q[BLOCKSIZE];
14     short max;
15     short min;
16     short sharedExponent;
17 }BFP;
18
19 typedef struct{
20     long d[3];
21     int d2[1];
22 }cBFP;
23
24 short arrMax(short *arr){
25     short max = arr[0];
26     for(int i = 1; i<BLOCKSIZE;i++){
27         if(max < arr[i])
28             max = arr[i];
29     }
30     return max;
31 }
32 short arrMin(short *arr){
33     short min = arr[0];
34     for(int i = 1; i<BLOCKSIZE;i++){
35         if(min > arr[i])
36             min = arr[i];
37     }
```

```

38     return min;
39 }
40 short max(short a, short b){
41     if(a >= b)
42         return a;
43     return b;
44 }
45 short min(short a, short b){
46     if(a <= b)
47         return a;
48     return b;
49 }
50 short compression(BFP *block){
51     for(int k = 0; k<NUMBER_OF_BLOCKS;k++){
52         block[k].max = max(arrMax(block[k].Im), arrMax(block[k].Q));
53         block[k].min = min(arrMin(block[k].Im), arrMin(block[k].Q));
54
55         short maxValue = max(block[k].max, abs(block[k].min) -1);
56
57         short rawExp = floor(log2(maxValue) +1);
58
59         short SharedExponent = max(rawExp - IQWIDTH +1, 0);
60
61
62         for( int i= 0 ; i<BLOCKSIZE;i++){
63             block[k].Im[i] = block[k].Im[i] >> SharedExponent;
64             block[k].Q[i] = block[k].Q[i] >> SharedExponent;
65         }
66         block[k].sharedExponent = SharedExponent;
67     }
68 }
69 void decompress(BFP *block, cBFP *compBlock){
70     for(int k = 0; k<NUMBER_OF_BLOCKS;k++){

```

```

71     for( int i= 0 ; i<BLOCKSIZE;i++){
72         block[k].Im[i] = block[k].Im[i] << block[k].sharedExponent;
73         block[k].Q[i] = block[k].Q[i] << block[k].sharedExponent;
74     }
75 }
76 }
77
78 int main(){
79     int bufSize = 12;
80     BFP Block[NUMBER_OF_BLOCKS];
81     cBFP compBlock[NUMBER_OF_BLOCKS];
82
83     for(int k = 0; k< NUMBER_OF_BLOCKS; k++){
84         for(int i = 0 ; i<BLOCKSIZE;i++){
85             int tmp;
86             scanf("%d", &tmp);
87             Block[k].Im[i] = (tmp & 0xFFFF0000) >> 16;
88             Block[k].Q[i] = tmp & 0x0000FFFF;
89         }
90     }
91     if(DEBUG_DATA){
92         printf("input data\n");
93         for(int k = 0; k<NUMBER_OF_BLOCKS;k++){
94             for(int i = 0 ; i < BLOCKSIZE;i++){
95                 printf("data row Im, %d ,Q %d \n", Block[k].Im[i], Block[k].Q[i] );
96             }
97             puts("");
98         }
99     }
100     struct timespec start1,end1, start2, end2;
101     long compTimes[ITERATIONS], reverseTimes[ITERATIONS];
102
103     for ( int j = 0 ; j < ITERATIONS;j++){

```

```

104     clock_gettime(CLOCK_MONOTONIC, &start1);
105     compression(Block);
106     clock_gettime(CLOCK_MONOTONIC, &end1);
107     compTimes[j]= (end1.tv_sec - start1.tv_sec)*1000000000L +
108     (end1.tv_nsec -start1.tv_nsec);
109
110
111     clock_gettime(CLOCK_MONOTONIC, &start2);
112     decompress(Block, compBlock);
113     clock_gettime(CLOCK_MONOTONIC, &end2);
114     reverseTimes[j]= (end2.tv_sec - start2.tv_sec)*1000000000L +
115     (end2.tv_nsec -start2.tv_nsec);
116 }
117
118 if(DEBUG_DATA){
119     printf("after de compression\n");
120     for(int k = 0; k<NUMBER_OF_BLOCKS;k++){
121         for(int i = 0 ; i < BLOCKSIZE;i++){
122             printf("data row Im, %d ,Q %d \n", Block[k].Im[i], Block[k].Q[i] );
123         }
124         printf("shared exponent is : %d\n", Block[k].sharedExponent);
125         puts("");
126     }
127 }
128 puts("Single Thread CPU");
129 for(int i = 0;i<ITERATIONS;i++){
130     printf("%ld ", compTimes[i]);
131 }
132 puts("");
133 for(int i = 0;i<ITERATIONS;i++){
134     printf("%ld ", reverseTimes[i]);
135 }

```


136 }

Listning 4.1: Kodvariant för BFP på CPU seriellt

4.3 CUDA

CUDA-program är i grunden ganska lika C-program, men man måste ta hänsyn till att programmet fortfarande exekveras delvis på CPU, medan vissa specifikt för-definierade funktioner kan utföras på GPU, som har potential att arbeta snabbare tack vare parallellisering över flera kärnor.

De senare varianterna av våra CUDA-program kommer att innehålla några av raderna från kodlistning 4.2. Kodsnutten visar användning av *threadIdx.x*, *y*, *blockIdx.x* och *blockDim.x*. Dessa strukturer har tre attribut: *x*, *y* och *z*, och används för att hantera indexering samt säkerställa att en specifik tråd arbetar med rätt data. Variablerna *index* och *column* får samma värde som trådens ID. Variabeln *PFB_index* får sitt värde utifrån blockets ID multiplicerat med tråddimensionens *z*, adderat med trådens *z*-index. Mer konkret, i den sista iterationen av CUDA-programmet har *z*-dimensionen storleken fyra, vilket innebär att det i det första CUDA-blocket ska finnas trådar för fyra PRBer. Dessa får sitt index utifrån vilket *z*-ID tråden har. Vid en midbandskörning finns det 546 CUDA-block, där varje block innehåller 96 trådar för en symbol.

```
1  int index = threadIdx.x;
2  int colum = threadIdx.y;
3  int PFB_index = blockIdx.x * blockDim.z + threadIdx.z;
```

Listning 4.2: Exempel på trådhanteringen inom CUDA

4.4 Seriell exekvering

Detta var det första program vi skapade med CUDA. CUDA-programmet liknar C-versionen, men innehåller vissa nyckelord och principer som normalt inte används i C/C++. Nyckelorden `__device__` och `__global__` är specifika för CUDA och används för att ange vilka funktioner som ska exekveras på grafikkortet. Kodlistning B.1 har CUDA anrop för att överföra minnet mellan host och device. Data vi har är lagrad i en lista av den *struct* som vi skapade kallad BFP som innehåller två listor för I och Q data som är tolv element lång, den innehåller max, min och den gemensamma exponenten.

```
1  __global__ void decompress(BFP *block){
2      int index = threadIdx.x;
3      int colum = threadIdx.y;
4      for(int k = 0; k<NUMBER_OF_BLOCKS;k++){
5          for(int i = 0; i<ROWS;i++){
6              block[k].data[i][0] = block[k].data[i][0] << block[k].sharedExponent;
7              block[k].data[i][1] = block[k].data[i][1] << block[k].sharedExponent;
8          }
9      }
```

Listning 4.3: Exempel på GPU seriell exekvering för dekomprimera

4.5 Parallell exekvering i ett PRB-block

Den första optimeringen på GPU är parallell exekvering inom ett PRB, där varje element i ett PRB-block ska bearbetas samtidigt. Dock sker inte allt parallellt. Den första delen av koden exekveras fortfarande seriellt. I Kodlistning B.2 kan man se att beräkningen av den gemensamma exponenten görs i en tråd, eftersom detta värde sedan används av övriga trådar för att bitförskjuta resterande element.

```
1  __global__ void compression(BFP *block){
2      int index = threadIdx.x;
3      int colum = threadIdx.y;
4      int PFB_index = blockIdx.x * blockDim.z + threadIdx.z;
5
6      if(index== 0){
7          block[PFB_index].max = arrMax(block[PFB_index].data);
8          block[PFB_index].min = arrMin(block[PFB_index].data);
9
10         short maxValue = max(block[PFB_index].max, abs(block[PFB_index].min) -1);
11
12         short rawExp = floor(log2(maxValue) +1);
13
14         short SharedExponent = max(rawExp - IQWIDTH +1, 0);
15         block[PFB_index].sharedExponent = sharedExponent;
16     }
17     __syncthreads();
18     for(int i = 0; i<NUMBER_OF_BLOCKS;i++){
19         block[PFB_index].data[i][0] = block[PFB_index].data[i][0] >>
20         block[PFB_index].sharedExponent;
21         block[PFB_index].data[i][1] = block[PFB_index].data[i][1] >>
22         block[PFB_index].sharedExponent;
23     }
24 }
```

Listning 4.4: Kodexempel på Warp divergent

Kodmässigt skiljer sig programmet i princip bara med att man tar bort en for-loop se exemplet i Kodlistning 4.4. I kodlistningen hanterar trådarna jobbet av att bitskifta elementen i listan. I exemplet ser man även att datastrukturen inte längre är 2 olika listor utan en matris som innehåller samma mängd element. Denna ändring beror på att vi använder trådarnas index som index i datastrukturen.

För att möjliggöra detta måste datan ligga i en matrisstruktur. CUDA-anropet i main ser också annorlunda ut, eftersom vi nu använder ett *dim3()*-anrop med argumenten 12, 2, 1. Detta skapar ett CUDA-block med totalt 24 trådar, fördelade över dimensionerna: 12 i x-led och 2 i y-led. Det är dessa index som används, och det är de trådarna som utför själva beräkningen.

```
1  __global__ void decompress(BFP *block){
2      int index = threadIdx.x;
3      int colum = threadIdx.y;
4      for(int k = 0; k<NUMBER_OF_BLOCKS;k++){
5          block[k].data[index][colum] = block[k].data[index][colum]
6          << block[k].sharedExponent;
7      }
8  }
```

Listning 4.5: GPU parallell inuti ett PRB dekomprimering exempel

4.6 Parallell exekvering över alla PRB block

Andra optimeringen är att använda en tråd för varje block. Precis som i kapitlet 4.5 ersätts en for-loop med trådar som arbetar var och en på var sitt eget PRB. I Kodlistning 4.6 liknar programmet det tidigare, men skillnaden ligger i var for-loopen är placerad. Istället för att ha en for-loop som itererar över alla block i hela programmet, finns nu en loop med 12 iterationer som hanterar själva komprimeringen. Denna förändring fördelar arbetsbelastningen jämnare mellan trådarna och utnyttjar fler kärnor.

```
1  __global__ void compression(BFP *block){
2      int index = threadIdx.x;
3      int colum = threadIdx.y;
```

```

4     int PFB_index = blockIdx.x * blockDim.z + threadIdx.z;
5     if(index== 0){
6         block[PFB_index].max = arrMax(block[PFB_index].data);
7         block[PFB_index].min = arrMin(block[PFB_index].data);
8         short maxValue = max(block[PFB_index].max, abs(block[PFB_index].min) -1);
9         short rawExp = floor(log2(maxValue) +1);
10        short SharedExponent = max(rawExp - IQWIDTH +1, 0);
11        block[PFB_index].sharedExponent = sharedExponent;
12    }
13    __syncthreads();
14    for(int i = 0; i<NUMBER_OF_ROWS;i++){
15        block[PFB_index].data[i][0] = block[PFB_index].data[i][0] >> block[PFB_index].sharedExponent;
16        block[PFB_index].data[i][1] = block[PFB_index].data[i][1] >> block[PFB_index].sharedExponent;
17    }
18 }
19

```

Listning 4.6: Exempel på parallell komprimering på GPU block

4.7 En tråd per element

Kodlistning B.4 är en kombination av tidigare program där inga for-loopar är kvar. Vi implementerade även med andra CUDA anrop som Pre-launching av funktioner, pinned memory, balans mellan trådar och CUDA block och även SM se Kodlistning 4.7 för ett kort exempel från programmet.

```

1     BFP *Block;
2     BFP *d_Block;
3     cudaMallocHost((void **)&Block, sizeof(BFP) * NUMBER_OF_BLOCKS * 14 * NUMBER_OF_MAST);
4     cudaMalloc((void **)&d_Block, sizeof(BFP) * NUMBER_OF_BLOCKS * 14 * NUMBER_OF_MAST);
5

```

```

6      .
7      .
8      .
9      cudaLaunchKernel((void *)compression, 1, 1, NULL,0,0);
10     cudaDeviceSynchronize();

```

Listning 4.7: GPU exempel på minnesallokering

Vi implementerade en lösning som bearbetar 14 symboler simultant för att möjliggöra prestandamätningar vid olika antal tidsslotar samt för att utnyttja fler SM. Detta fördelar arbetsbelastningen över flera streams, vilket illustreras i Kodlistning 4.8. I exemplet används for-loopar för att iterativt anropa CUDA-funktioner genom CUDA API:t, där varje anrop behandlar en specifik delmängd av data.

```

1      for ( int j = 0 ; j <= ITERATIONS / (14*NUMBER_OF_MAST);j++){
2          for(int k = 0; k < 14 * NUMBER_OF_MAST; k ++ )
3              readData(Block+ (NUMBER_OF_BLOCKS * k), FP, NUMBER_OF_BLOCKS);
4
5              clock_gettime(CLOCK_MONOTONIC, &start1);
6          for(int k = 0 ; k < NUMBER_OF_STREAMS * NUMBER_OF_MAST; k++){
7              if (j * 14 + k == 100)
8                  break;
9              cudaMemcpyAsync(d_Block + (NUMBER_OF_BLOCKS * k), Block + (NUMBER_OF_BLOCKS * k),
10                 DataAmount , cudaMemcpyHostToDevice, streams[k]);
11          }
12
13
14          for(int k = 0 ; k < NUMBER_OF_STREAMS * NUMBER_OF_MAST; k++)
15              compression<<<CUDA_BLOCK ,dim3(12,2,ZTHREADS),0,streams[k]>>>
16              (d_Block + (NUMBER_OF_BLOCKS * k));
17
18          for(int k = 0 ; k < NUMBER_OF_STREAMS * NUMBER_OF_MAST; k++)

```

```
19         cudaMemcpyAsync(compblock + (NUMBER_OF_BLOCKS * k), d_block +  
20         (NUMBER_OF_BLOCKS * k),  
21         CompDataAmount, cudaMemcpyDeviceToHost, streams[k]);
```

Listning 4.8: Kodexempel på hur man kan använda flera streaming multiprocessors

Kapitel 5

Resultat

Inledningsvis i detta kapitel beskrivs de målen och resultaten av arbetet. Följt av är resultaten på de CUDA program vi skrev. Slutligen utvärderas arbetet och resultatet.

5.1 Mål och resultat

Huvudmålet med arbetet är att verifiera om användningen av GPU för BFP-algoritmen är utförbart och rimligt givet de resurser som finns tillgängliga. För att fastställa om BFP-algoritmen uppfyller dessa krav testades de program vi utvecklade.

En faktor som kan bidra till lägre prestanda än förväntat är overhead, vilket avser den extra tid och resursförbrukning som uppstår utöver den faktiska beräkningen. Overhead kan exempelvis bero på hantering av trådar eller initialisering av beräkningskärnor. För att kunna analysera hur olika förbättringar påverkar prestandan planeras utveckling av flera programvaruimplementationer, med syftet att kvantifiera den prestandaförbättring som varje steg ger.

Som referens utvecklas även ett seriellt program i C som körs på CPU. Exekveringstiden för detta program utgör en baslinje och uppgår till cirka 120 μ s för komprimering av en symbol. De exekveringstider som presenteras i resultatkapitlet baseras

huvudsakligen på medelvärden från 100 upprepade mätningar, i syfte att säkerställa en representativ och pålitlig bild av prestandan. Utöver medelvärdet är det också av intresse att undersöka om exekveringstiden i enskilda fall överskrider definierade tidsgränser, då detta kan ha betydelse för systemets tillförlitlighet i realtidsapplikationer.

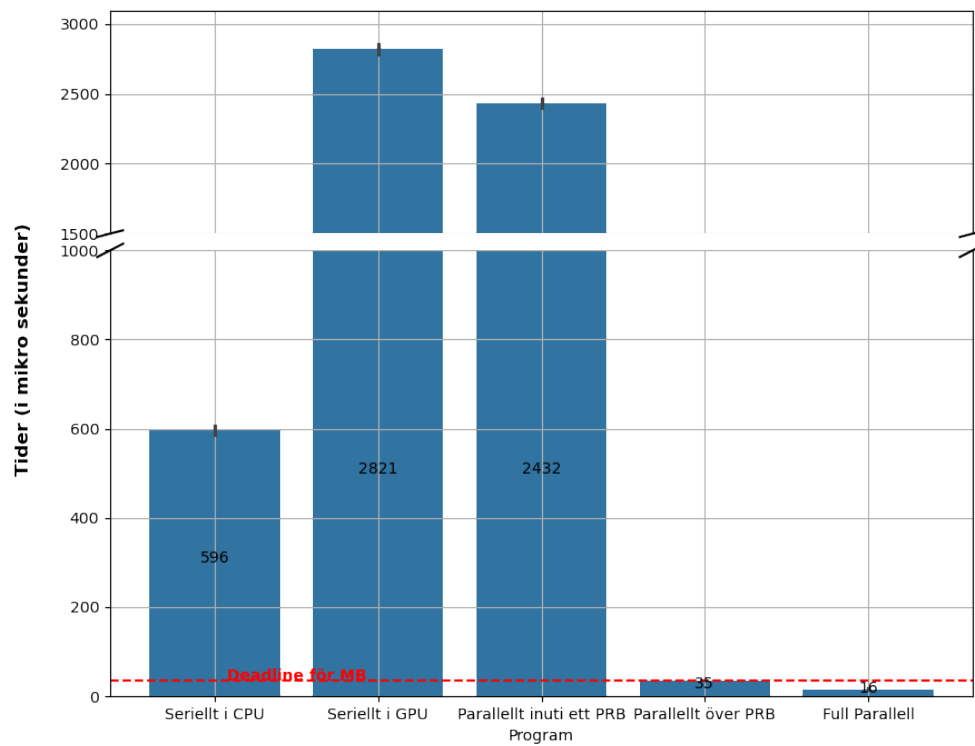
För att ge en översikt av programmets prestanda och dess förbättringar presenteras två stapeldiagram. På x-axeln visas de fem olika iterationerna av programmet, där varje stapel representerar medelvärdet från 100 körningar. Staplarna är försedda med 95 % konfidensintervall, markerade med svarta streck. Y-axeln visar exekveringstider i mikrosekunder, från 0 till 1000, med ett avbrott, och fortsätter därefter från 1500 till 3000 mikrosekunder. En röd streckad linje markerar deadline för midband.

I fig. 5.1 exekverade det första programmet en symbol på 596 mikrosekunder, det andra på 2821 mikrosekunder, det tredje på 2432 mikrosekunder, det fjärde på 35 mikrosekunder och det femte på 16 mikrosekunder.

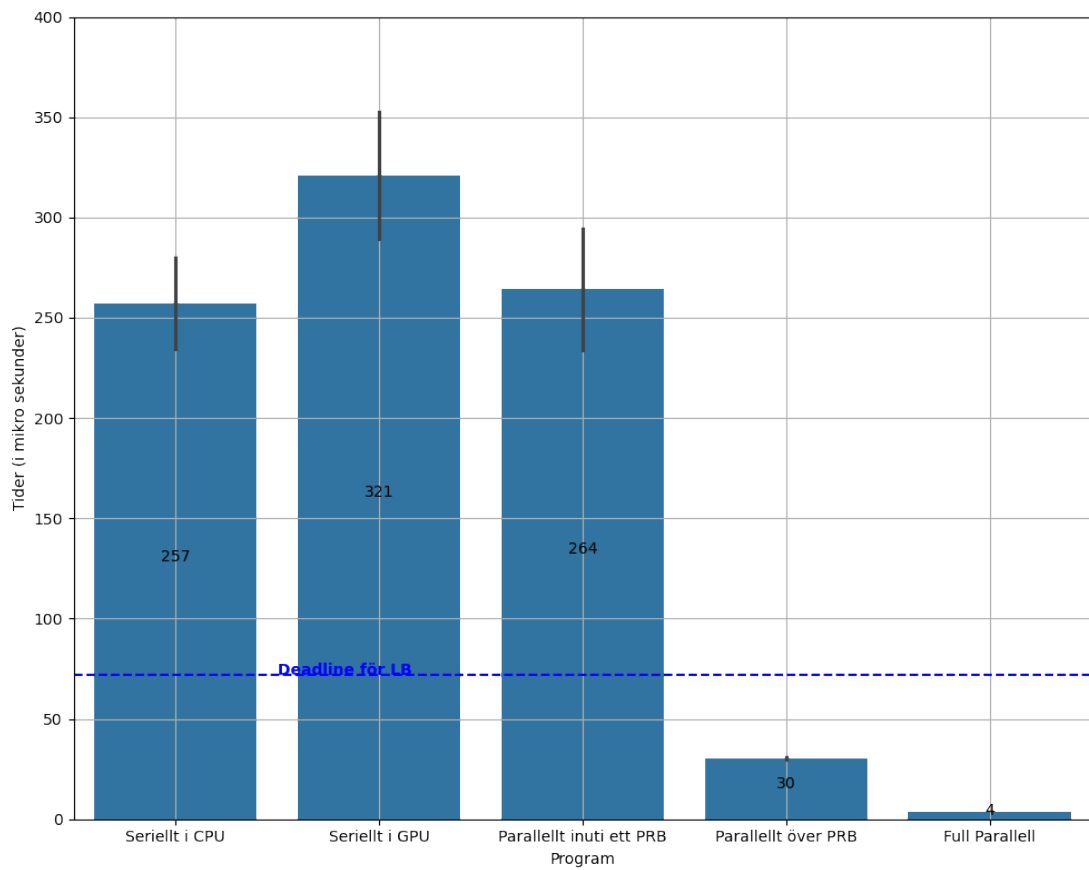
I fig. 5.2 visas samma programversioner fast för lågband på x-axeln och exekveringstider på y-axeln, denna gång mellan 0 och 400 mikrosekunder. En blå streckad linje markerar deadline för lågband. Det första programmet exekverade en symbol på 257 mikrosekunder, det andra på 321 mikrosekunder, det tredje på 264 mikrosekunder, det fjärde på 30 mikrosekunder och det femte på 4 mikrosekunder.

Resultaten visar tydligt hur varje iteration på GPU förbättrade exekveringstiden, och att de två sista versionerna har realistiska möjligheter att klara respektive deadline.

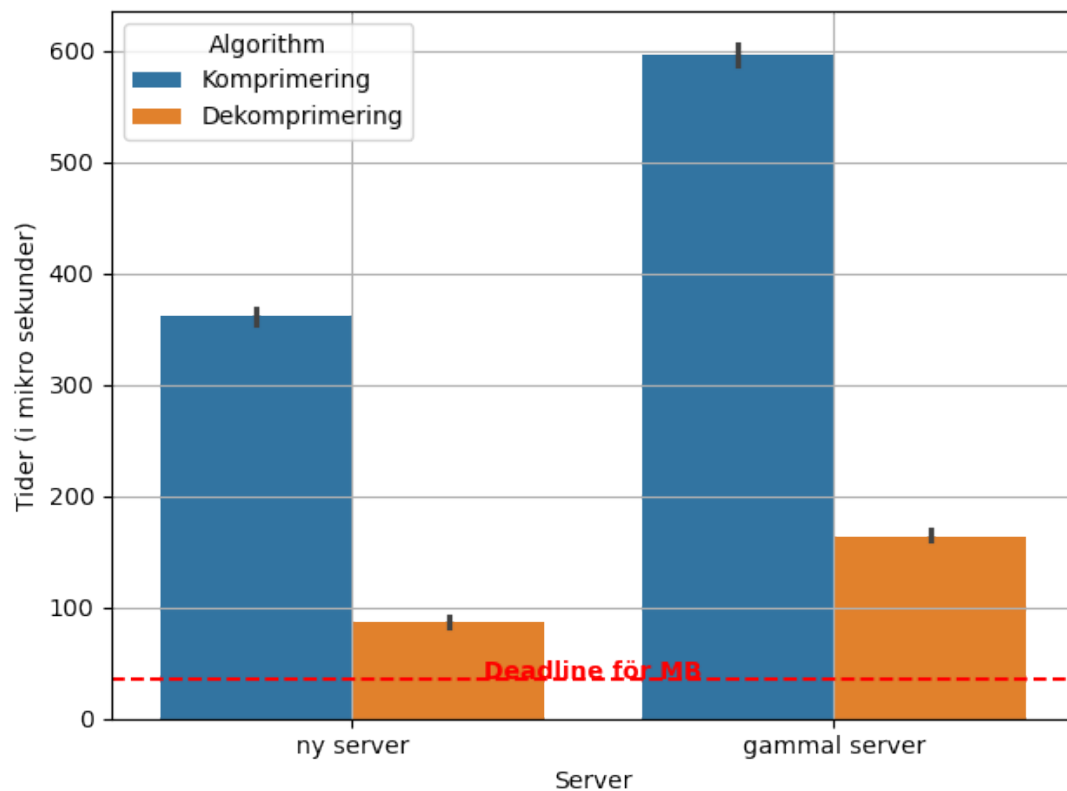
Figur 5.3 visar hur lång tid det tog att komprimera samt dekomprimera en symbol för vår referensimplementering i C. I figuren visas tider både för den gamla och den nya servern. Y-axeln visar den tid det tog i mikrosekunder och x-axeln visar en stapel för specifik server och algoritm. Varje stapel representerar ett medelvärde av 100 stycken repetitioner. För varje stapel finns även ett 95% konfidensintervall som illustreras av ett svart streck. Här illustreras att den nya servern får en tid på cirka 350 mikrosekunder på komprimering och cirka 80 mikrosekunder för dekomprimering. Den gamla servern



Figur 5.1: Stapeldiagram över exekveringstiderna för programversionerna (tider per symbol på midband på RTX3090).



Figur 5.2: Stapeldiagram över tiderna på programmen använt under arbetet(tider för en symbol på lågband på RTX3090)



Figur 5.3: Stapeldiagram över komprimering för en symbol på CPU (en symbol)

har medelvärdestider på respektive 600 mikrosekunder och 160 mikrosekunder. Detta kan inte ses ifrån detta stapeldiagram men fördelningen av tider är höger skev. För kommande resultat kommer liknande grafer att användas.

5.2 CUDA program

I denna sektion redovisas resultatet av programmen vi skrev i CUDA. Resultatet här är det som är i huvudfokus, att optimera med hjälp av GPU-hårdvara.

5.2.1 Exekvering på en kärna

C-programmet för komprimering översattes till CUDA. Det resulterande programmet hade längre exekveringstid. Att ett seriellt program generellt tog längre tid att exekvera på GPU var förväntat, eftersom klockfrekvensen hos en GPU är lägre än hos en CPU.

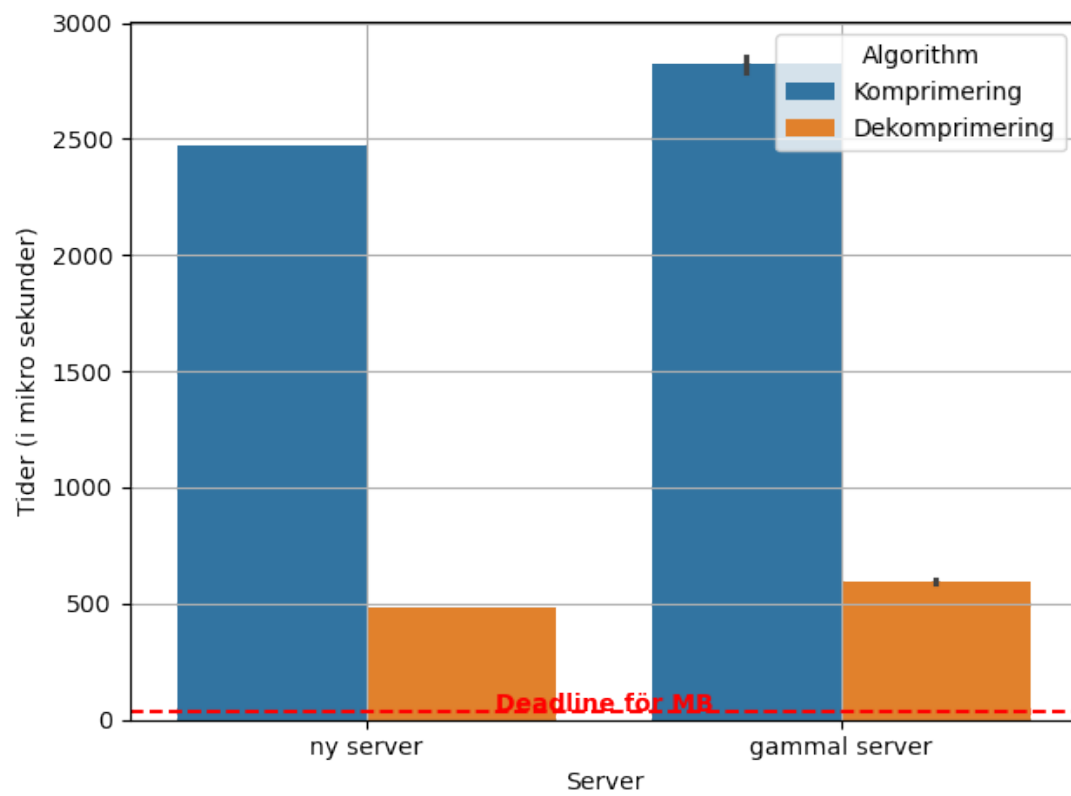
Observera i Kodlistning B.1 att programstrukturen i stort sett är likartad, men att CUDA kräver särskild syntax för att specificera vilka funktioner som ska exekveras på GPU.

I Figur 5.4, likt i den tidigare grafen, illustreras genomsnittliga tider för komprimering och dekomprimering av en symbol i midband på den gamla respektive den nya servern. Y-axeln representerar tid i mikrosekunder, medan x-axeln visar staplar för varje kombination av server och algoritm. Den nya servern uppvisar i genomsnitt strax under 2500 mikrosekunder för komprimering och cirka 500 mikrosekunder för dekomprimering. Den gamla servern ligger närmare 2750 mikrosekunder vid komprimering och omkring 600 mikrosekunder vid dekomprimering. Tiderna för GPU är avsevärt längre än för CPU; dessutom är dessa värden uppmätta utan att överföringstiden via PCIe har inkluderats.

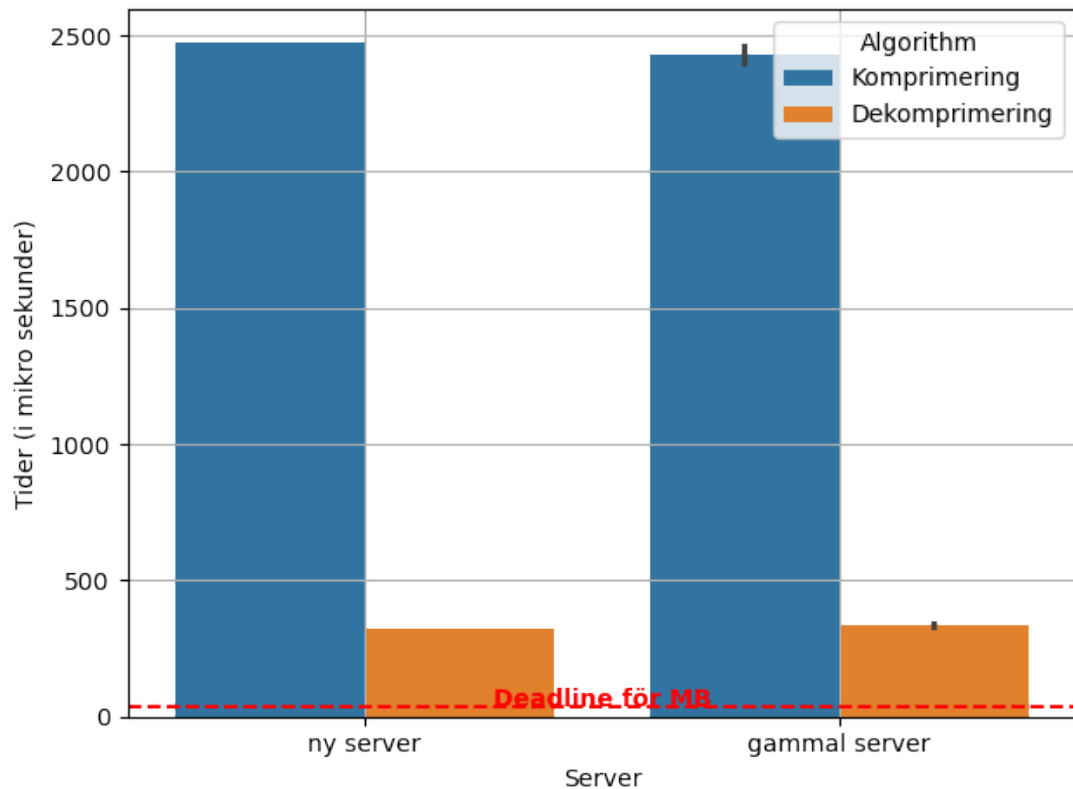
5.2.2 Parallell exekvering internt i ett block

Programmet i kodlistning B.2 exekverades parallellt för alla element inuti ett PRB, men körde 2184 PRB i serie. Varje element hanterades av en tråd. Denna variant exekverade dock inte alla operationer parallellt inom ett PRB, eftersom beräkningen av den delade exponenten krävde att både minimum, maximum och ett logaritmiskt värde av det största beloppet i blocket först beräknades.

I Figur 5.5, likt i tidigare diagram, illustreras genomsnittliga tider för komprimering och dekomprimering av en symbol i midband på den gamla respektive den nya servern. Y-axeln representerar tid i mikrosekunder, medan x-axeln visar en stapel för varje kom-



Figur 5.4: Stapeldiagram för seriell exekvering för GPU (en symbol)



Figur 5.5: Stapeldiagram för parallel exekvering intärnt i ett PRB med GPU (en symbol)

bination av server och algoritm. Den nya servern, liksom det seriella programmet på GPU, hamnar runt 2500 mikrosekunder för komprimering och cirka 500 mikrosekunder för dekomprimering. Den gamla servern uppvisar även den tider strax under 2500 mikrosekunder för komprimering och cirka 600 mikrosekunder för dekomprimering.

Tiderna för parallell exekvering kan inte beskrivas som en kvot av seriell tid genom antalet trådar.

$$\frac{T_0}{\#Threads} \neq T_1$$

Detta syns tydligt i detta program som har 24 trådar, vilket borde ha en avsevärt mindre exekveringstid. Men man får ta hänsyn till hur kärnorna i GPU jobbar och eventuellt overhead.

5.2.3 Parallell exekvering över alla block

Blocken kördes parallellt men hade enbart en tråd inuti varje PRB vilket kunde arbeta på alla block parallellt men körde innehållet i blocket seriellt.

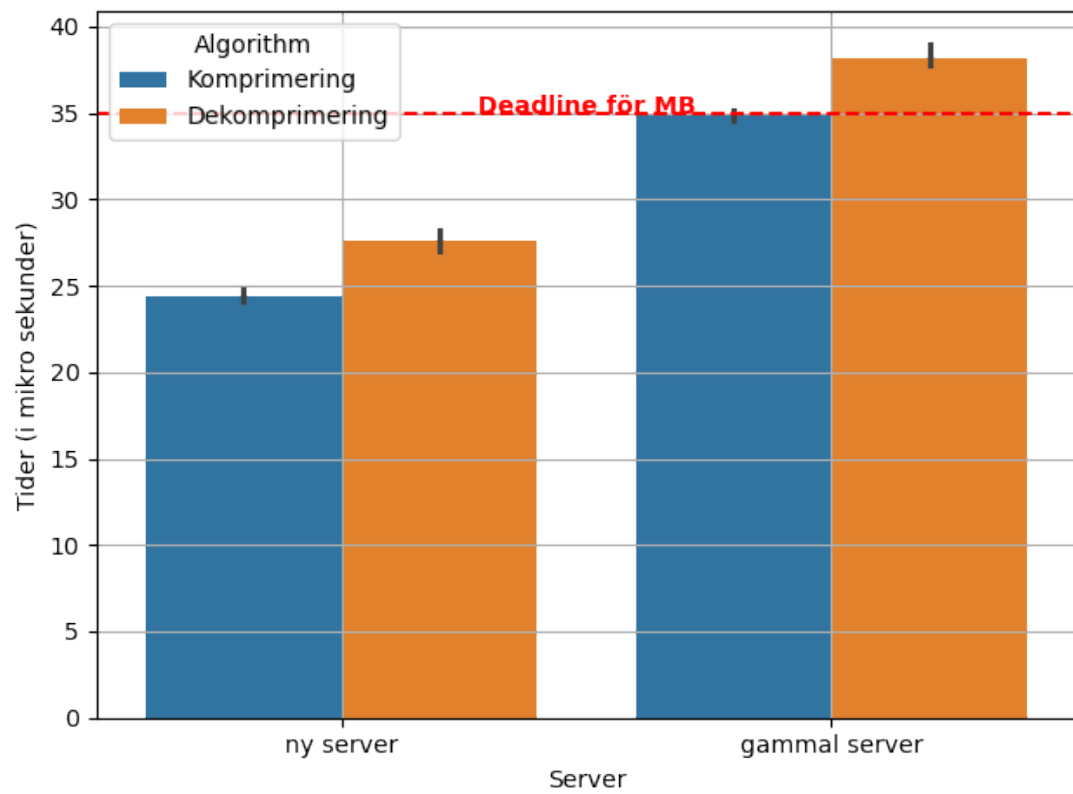
Figur 5.6 likt den tidigare grafen illustreras genomsnittliga tider för att komprimera och dekomprimera för en symbol på midband på den gamla och nya servern. I figuren representerar y-axeln tid i mikrosekunder och x-axeln visar en stapel för varje mätning med server och vilken algoritm. Den nya servern hamnar runt 24 mikrosekunder i komprimering och cirka 27 mikrosekunder på dekomprimering. Den gamla servern ligger på cirka 35 mikrosekunder i komprimering och 37 mikrosekunder i dekomprimering.

Programmet exekverar med 2184 kärnor det bör teoretiskt bli max 2184 gånger snabbare men dessa är runt 350 gånger snabbare än seriellt på GPU.

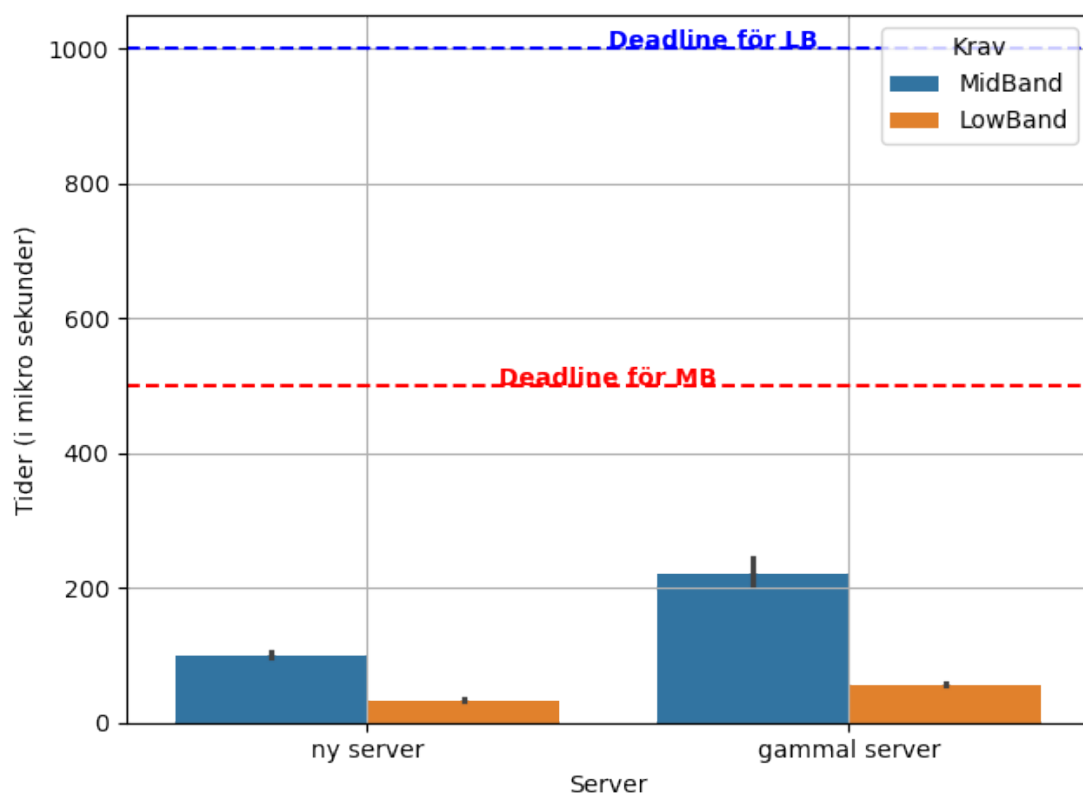
5.2.4 Parallell exekvering med en tråd för varje element

Denna variant tilldelade en tråd för varje I- och Q-element, vilket möjliggjorde att en kärna kunde bearbeta ett element. Det visade sig dock att exekveringstiderna delvis förbättrades jämfört med referensprogrammet. En hastighetsförbättring på ungefär en faktor två observerades i jämförelse med den fjärde iterationen för en symbol. Om varje tråd verkligen hade tilldelats en dedikerad kärna skulle man, vid ideal parallellisering, kunna förvänta sig en acceleration på upp till 24 gånger. Med hänsyn till overhead såsom synkronisering, minnesåtkomst och övrig kommunikation är det dock inte realistiskt att uppnå denna teoretiska maxprestanda.

Figur 5.7, i likhet med tidigare diagram, visar genomsnittliga exekveringstider för komprimering. I detta fall avser mätningen dock en hel tidslot, snarare än en enskild symbol, i både midband och lågband på den gamla och den nya servern. Y-axeln anger tid i mikrosekunder, och x-axeln visar staplar för varje kombination av server och algoritm. Den nya servern uppvisar komprimeringstider strax under $100\mu s$ i midband och



Figur 5.6: Stapeldiagram för parallel exekvering över alla block på GPU (en symbol)



Figur 5.7: Stapeldiagram över komprimering för en tidsslot på GPU.

cirka $25\mu s$ vid dekomprimering. På den gamla servern ligger motsvarande tider kring $220\mu s$ för komprimering i midband och cirka $50\mu s$ i lågband.

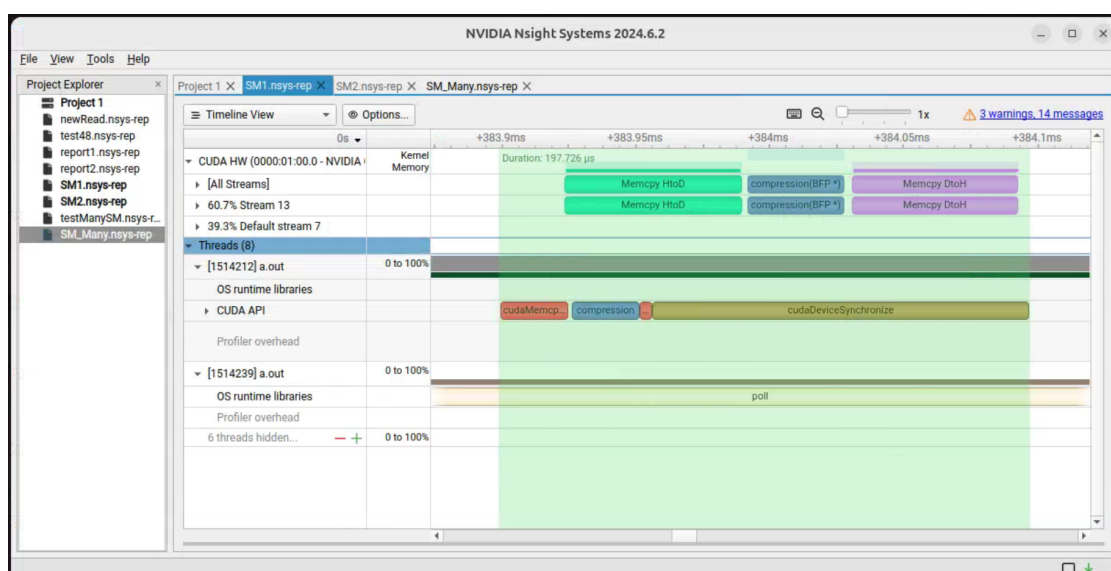
Den femte iterationen av vårt program, som beskrivs i kapitel 4 utökades för att exekveras med flera CUDA-streams och tidsslotar. Syftet med denna förändring var att ytterligare förbättra graden av parallellism i exekveringen. Det kunde dock konstateras att systemets huvudsakliga flaskhals kvarstod i form av minnesöverföring, vilken upptog majoriteten av den totala exekveringstiden, trots att beräkningarna utfördes parallellt.

Denna tidsmätning kör 1 SM för en tidsslot. Observera att Figur 5.8 innehåller dubbla uppsättningar gröna, blåa och lila rutor som representerar CUDA-anrop. Den översta raden, med titeln All streams, summerar exekveringen av samtliga streams i en gemensam tidslinje. Syftet med denna vy är att ge en helhetsbild av parallell exekvering,

vilket är särskilt värdefullt när ett stort antal streams används, annars blir det svårt att identifiera enskilda anrop.

Denna körning tog cirka 198 μs i Round Trip Time (RTT), vilket inkluderar både överföring av data till och från GPU samt initial latens från det första anropet till att den första CUDA-kärnan startar. Programmet visar tydligt att en betydande andel av exekveringstiden består av minnesöverföring. Mätningar visade en överföringshastighet på cirka 22GiB/s till GPU och 23GiB/s från GPU, vilket är relativt lågt i förhållande till den teoretiska bandbredden hos PCIe 5.0 x16, som uppgår till 64GiB/s i vardera riktningen.

I Figur 5.8 visas även ett grafiskt exempel på hur exekveringen för den sista iterationen förlöpte över tid.

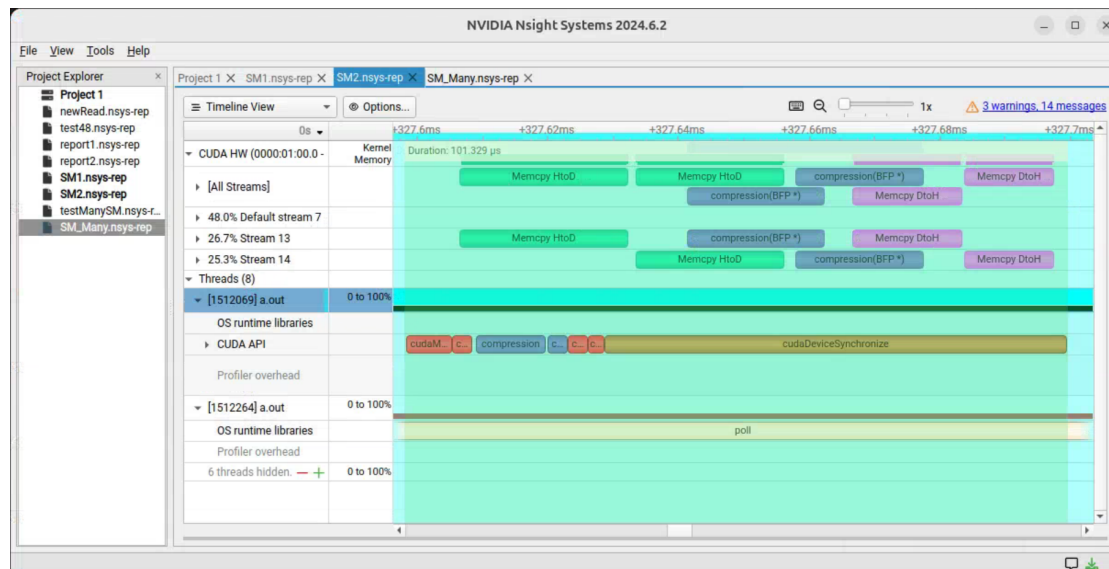


Figur 5.8: Nsight system mätning över 1 stream

I Figur 5.9 visas ett dubbelt antal block jämfört med den tidigare figuren. Varje stream i GPU:n tilldelas hälften av arbetslasten som i det föregående exemplet. Detta möjliggör för stream 13 att avbryta sin minnesöverföring tidigare och påbörja exekveringen av algoritmen.

Den totala tiden för denna mätning är strax över 101 mikrosekunder nästan hälften av tiden i det tidigare exemplet och uppnås genom att en ytterligare SM används i programmet.

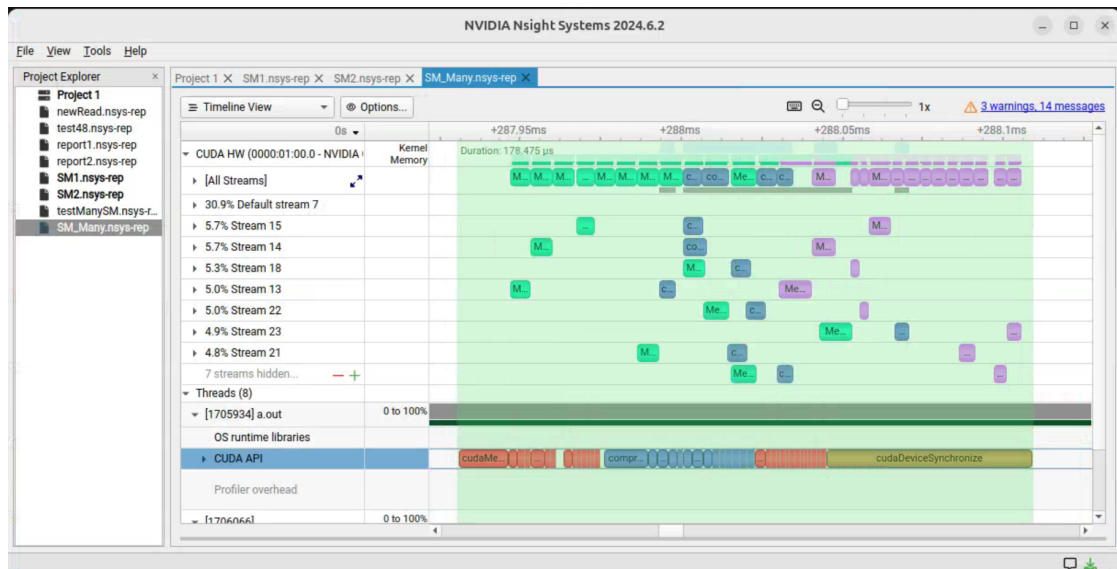
Förutom att kärnorna kan börja exekvera tidigare, kan dataöverföring ske under en större del av programmets gång och samtidigt med exekveringen.



Figur 5.9: Nsight system mätning över 2 stream

I detta exempel, Figur 5.10, används 14 CUDA-streams. Detta borde teoretiskt sett resultera i det snabbaste programmet, eftersom fler kärnor är tillgängliga och det finns större potential för parallell hantering av operationer till och från GPU:n. I praktiken blir det dock inte så. Efter ett visst antal streams börjar prestandan istället att sjunka. Minnesöverföringsblocken blir långsammare, och antalet API-anrop på host-sidan ökar, vilket leder till att mer tid går åt till overhead. Detta påverkar prestandan negativt, eftersom tråden istället hade kunnat utnyttjas för annat arbete.

I programmet testade vi även möjligheten att låta en GPU komprimera och överföra data till flera celler. I varje cell finns en mast med antenner som sänder ut den komprimerade data i form av radiovågor. Tiden ökade linjärt eftersom dataöverföringen utgjorde



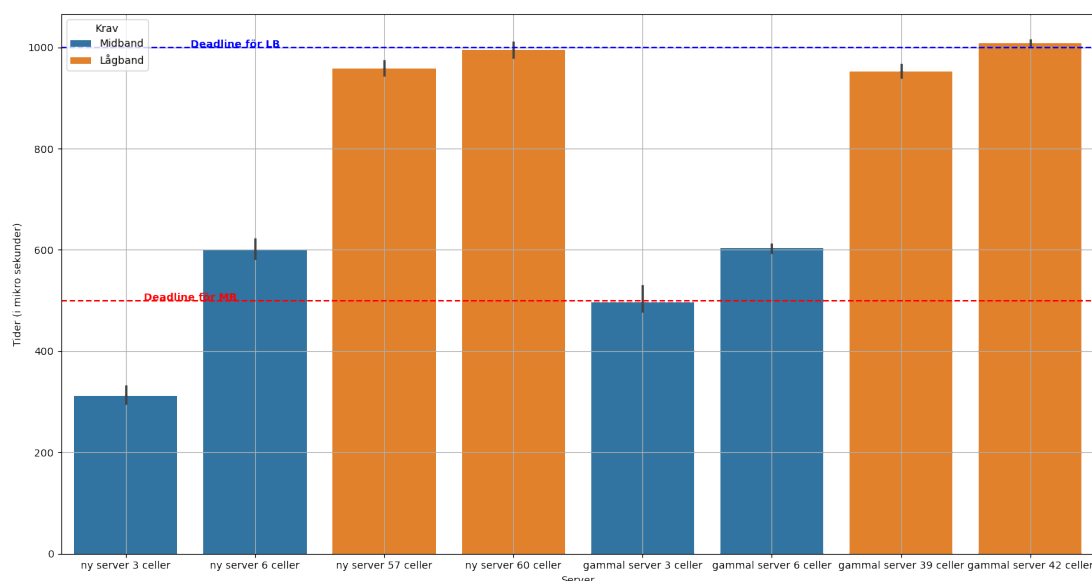
Figur 5.10: Nsight system mätning över 14 stream

flaskhalsen, och datan kunde inte bearbetas förrän den var helt överförd.

För midband gjordes tester för flera celler då intresset från arbetsgivare är att testa för en multipel av tre celler.

För lågband är deadline-tiden längre och antalet PRB per symbol lägre. Det är möjligt att köra många celler parallellt, men vi visar inte alla kombinationer av tider och antal SM i denna rapport. Under arbetets gång har vi kunnat fastställa att en fullt parallell användning av GPU:n, där alla symboler i en tidslucka exekveras samtidigt kan ge mycket snabb exekvering av BFP-algoritmen. Detta visar att en implementation med GPU är möjlig. Dock är en lösning som baseras på satsvis bearbetning inte alltid önskvärd, eftersom den introducerar latens genom att behöva vänta in data innan bearbetningen kan påbörjas.

De program som utvecklats har generellt blivit snabbare för varje iteration. Den senaste versionen, som kombinerar olika datamängder och antal trådar, uppnår en prestandaförbättring med en faktor mellan 50 och 60. Trots detta har programvaran som hanterar sex celler för midbandet fortfarande en för hög total exekveringstid.



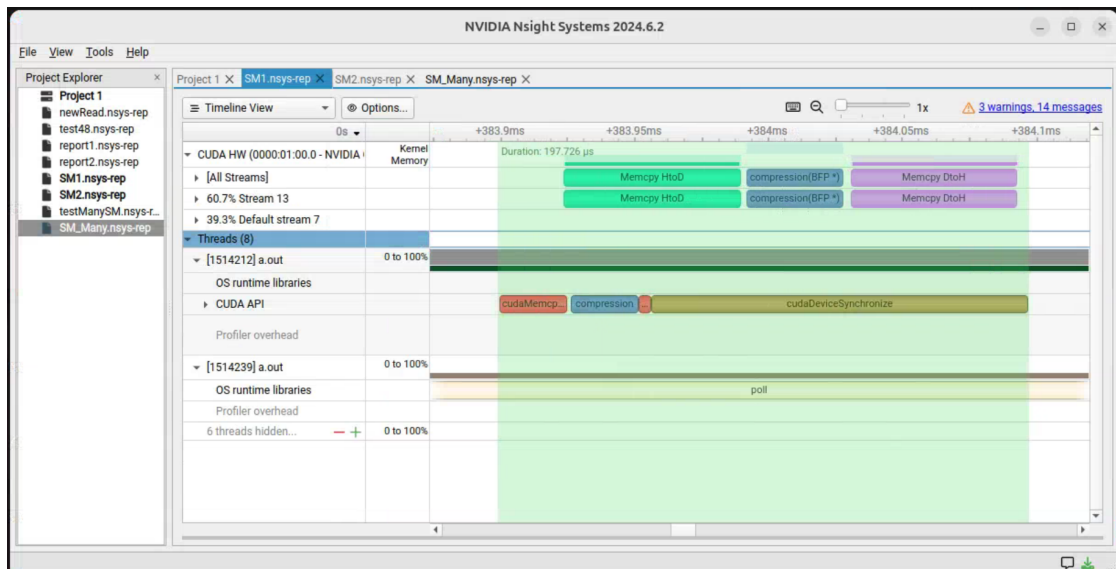
Figur 5.11: Stapeldiagram över körningar för olika antal celler.

Att delegera BFP-algoritmen till GPU har visat sig vara mindre effektivt än förväntat, då överföringen mellan CPU och GPU i hög grad begränsar prestandan. Överföringstiden kan utgöra upp till 60% av den totala exekveringstiden. I mätningar har dataöverföringen till och från GPU uppgått till 65 respektive 62 mikrosekunder.

I Figur 5.12, som visar en körning för midbandet under en slottid, framgår flera aspekter av exekveringen. Om man däremot fokuserar specifikt på de block där faktiskt arbete sker, uppgår överförings- och beräkningstiderna till 65, 36 och 62 mikrosekunder för respektive moment.

$$\frac{65\mu + 62\mu}{65\mu + 36\mu + 62\mu} = 78\%$$

Detta innebär att cirka 78% av exekveringstiden består av enbart dataöverföring, vilket är en betydande andel och en klar flaskhals i systemets prestanda.



Figur 5.12: Nsight system exempel på tider i en GPU körning

5.3 Utvärdering

Även med användning av tekniker såsom pinned memory, prelauncha kernels, parallella streams, asynkrona minnesöverföringar och en rimlig fördelning av data per stream, uppnås en optimal exekveringstid för en mast på cirka 110 mikrosekunder för en hel slot på RTX 5090. Detta motsvarar ungefär en femtedel av den satta deadline-tiden. Upp till 70% av dessa 110 mikrosekunder utgörs av dataöverföring via PCIe mellan host och device.

I Tabell 5.1 finns en lista över de program vi skapade och tiderna vi samlade vid körning och tidtagning på den gamla servern. I tabellen finns elva kolumner, den första *Komponenten* specificerar vilken hårdvara som användes för den tidtagningen. Den andra kolumnen *Algoritm* visar ifall tiden är för dekomprimering eller komprimering. Den tredje kolumnen *Krav* visar ifall det är för midband eller lågband och där indikerar hur mycket data som algoritmen exekveras på. Den fjärde kolumnen *Variant* specificerar iterationen av förbättring som användes. Den femte kolumnen *SM* är antalet SM som användes vid exekveringen. Den sjätte kolumnen *Celler* visar för hur många celler

Komponent	Algoritm	Krav	Variant	överföring	SM	Celler	slot T	symbol T	T/antenn	x
CPU	Komprimering	Midband	Seriell	FALSE	0	1	8260	590	73,75	1
CPU	Dekomprimering	Midband	Seriell	FALSE	0	1	2380	170	21,25	1
GPU	Komprimering	Midband	Seriell	FALSE	1	1	38500	2750	343,75	0,21
GPU	Dekomprimering	Midband	Seriell	FALSE	1	1	8680	620	77,5	0,27
GPU	Komprimering	Midband	parallell internt PRB	FALSE	1	1	33600	2400	300	0,25
GPU	Dekomprimering	Midband	parallell internt PRB	FALSE	1	1	5600	400	50	0,43
GPU	Komprimering	Midband	parallell över alla PRB	TRUE	1	1	483	34,5	4,31	17,11
GPU	Dekomprimering	Midband	parallell över alla PRB	TRUE	1	1	518	37	4,63	4,59
GPU	Komprimering	Midband	Parallell	TRUE	2	1	215	15,36	1,92	38,41
GPU	Komprimering	Lowband	Parallell	TRUE	2	1	57,7	4,12	1,03	71,6
GPU	Komprimering	Midband	Parallell	TRUE	2	3	480	34,29	4,29	17,19
GPU	Komprimering	Midband	Parallell	TRUE	1	6	614	43,86	5,48	13,46
GPU	Komprimering	Lowband	Parallell	TRUE	1	39	919,2	65,66	16,42	4,49
GPU	Komprimering	Lowband	Parallell	TRUE	1	42	1000,6	71,47	17,87	4,13

Tabell 5.1: Tabell över programmen tiderna för en tidsslot, en symbol och per antenn (gamla servern)

programmet kör samtidigt. Den sjunde kolumnen *slot T* visar tiden för en hel tidsslot, där en tidsslot är 14 symboler. Den åttonde kolumnen *symbol T* är tiden för enbart en symbol. Den nionde kolumnen *T/antenn* är tiden per antenn och beror på kravet för lågband som har fyra antenner medan midband har åtta antenner. Den sista kolumnen *x* indikerar hur mycket snabbare ett program exekverar jämfört med referensprogrammet på antenntiderna. Samma tabell finns även senare för 5090. Det vi ser är att tiderna per antenn blir bättre för varje optimering vi implementerar och den sista med två SM är den snabbaste varianten. Det går även att köra för flera celler samtidigt på denna server men det blir för lite marginal på midband. Lågband har generellt lägre krav och skulle kunna klara cirka 39 celler.

I Tabell 5.2, liksom i den tidigare tabellen, kan vi observera prestandaförbättringar för varje optimeringssteg. Den bästa prestandan uppnås med två SM, både för lågband och midband. Den nya servern uppvisar generellt kortare exekveringstider och hantlar beräkningarna mer effektivt. Den bäst presterande varianten för midband med två strömmar exekverar på den nya servern på ungefär halva tiden jämfört med den gamla.

När det gäller implementationen lyckades vi ta fram ett fungerande program som låg nära tidsgränsen för att bearbeta en symbol i midbandsläge på en RTX 3090. Vi utgick från en enkel struktur baserad på hur datan normalt är organiserad, vilket resulterade i

Komponent	Algoritm	Krav	Variant	överföring	SM	Celler	slot T	symbol T	T/antenn	x
CPU	Komprimering	Midband	Seriell	FALSE	0	1	5040	360	45	1
CPU	Dekomprimering	Midband	Seriell	FALSE	0	1	1120	80	10	1
GPU	Komprimering	Midband	Seriell	FALSE	1	1	34650	2475	309,38	0,15
GPU	Dekomprimering	Midband	Seriell	FALSE	1	1	6720	480	60	0,17
GPU	Komprimering	Midband	parallell internt PRB	FALSE	1	1	34608	2472	309	0,15
GPU	Dekomprimering	Midband	parallell internt PRB	FALSE	1	1	4480	320	40	0,25
GPU	Komprimering	Midband	parallell över alla PRB	TRUE	1	1	336	24	3	15
GPU	Dekomprimering	Midband	parallell över alla PRB	TRUE	1	1	364	26	3,25	3,08
GPU	Komprimering	Midband	Parallell	TRUE	2	1	110	7,86	0,98	45,92
GPU	Komprimering	Lowband	Parallell	TRUE	2	1	42	3	0,75	60
GPU	Komprimering	Midband	Parallell	TRUE	2	3	303	21,64	2,71	16,61
GPU	Komprimering	Midband	Parallell	TRUE	1	6	606	43,29	5,41	8,32
GPU	Komprimering	Lowband	Parallell	TRUE	1	57	950	67,86	16,97	2,65
GPU	Komprimering	Lowband	Parallell	TRUE	1	60	995	71,07	17,77	2,53

Tabell 5.2: Tabell över programmen tiderna för en tidsslot, en symbol och per antenn (nya servern)

att vi kunde använda cirka 96 av 128 kärnor per stream.

Det finns dock möjlighet till förbättring genom att strukturera programmet mer efter vad som är optimalt för hårdvaran, det vill säga på ett sätt som möjliggör användning av samtliga 128 kärnor. Utmaningen ligger i att hantera datan på ett sätt som begränsar varje tråd till endast det relevanta datablocket. Detta är betydligt mer komplicerat att implementera vid ett stort antal trådar, vilket är varför vi inte försökte med den implementeringen.

Kapitel 6

Slutsats

6.1 Diskussion

Telekomföretag som utvecklar 5G och 6G strävar efter att skapa system med högre stabilitet och låg latens i dataöverföring. En del i detta arbete är datakomprimering med olika tekniker, vilket i vårt fall är användning av BFP-algoritmen tillsammans med Nvidias GPU-hårdvara. Den data vi har samlat in tyder på att GPU-hårdvara inte är optimal för just BFP-algoritmen. Anledningen till det är att om man väljer att köra algoritmen på en GPU, utgör överföringen av data från CPU:n till GPU:n ett hinder när det gäller den totala exekveringstiden. I ett exempel visade en mätning att cirka 78 procent av exekveringstiden gick åt till just denna dataöverföring, vilket motverkar syftet med att använda GPU för att uppnå snabbare bearbetning än med CPU. Prestandan är inte tillräckligt god för att motivera att detta arbete delegeras till GPU. Dock kan det vara en möjlig lösning i scenarier där en basstation behöver mer processorkraft. I sådana fall kan man avlasta arbete från CPU till GPU, vilket totalt sett kan leda till en högre processeringskapacitet. En annan aspekt som bör nämnas är att själva BFP-algoritmen inte är särskilt komplex, vilket innebär att det inte krävs en avancerad setup som till exempel ett RTX 3090- eller 5090-kort för att köra den. Därför är en sådan investering

inte motiverad när man väger prestanda mot kostnad. Därför kan vi dra slutsatsen att den mest lämpliga hårdvaran för att köra BFP-algoritmen är CPU:n.

6.2 Sammanfattning

Resultaten från den insamlade data, som bearbetades genom vårt arbete, visar att GPU inte är den optimala hårdvaran för detta ändamål, främst på grund av att dataöverföringen tar för lång tid. Trots detta kan användning av GPU vara motiverad i vissa scenarier. Vid hantering av små datamängder framstår dock GPU som ett mindre lämpligt val för att köra BFP-algoritmen, eftersom en stor del av dess resurser förblir outnyttjade.

6.3 Utvärdering

Efter att ha utvärderat GPU-hårdvaran med parallell exekvering av BFP-algoritmen har vi fått en djupare förståelse för programspråket CUDA och hur man ska utnyttja hårdvarans resurser för att utföra ett arbete parallellt som BFP-algoritmen, samt hur GPU-arkitekturer är konstruerade för att möjliggöra parallell bearbetning, medan CPU-arkitekturer är mer lämpade för sekventiellt arbete. Dataöverföringen som finns i vårt arbete minskade avsevärt de totala prestandafördelarna med att använda GPU:n för beräkning. För att illustrera ett scenario där denna flaskhals inte är så stor kan man överväga GPU-baserade algoritmer som är mer process krävande till förhållande till dataöverföring från huvudtråden. I sådana fall kan fler SM användas och fördela upp beräkningen över flera kärnor. Detta scenario borde vara avsevärt bättre jämfört med en CPU-baserad implementering.

Uppdragsgivaren på Tietoevry är nöjd med det genomförda arbetet, men hade förväntat sig bättre resultat vid parallell exekvering, med tanke på att GPU-hårdvaran har ett stort antal kärnor jämfört med CPU:n.

6.4 Vidare arbete

De framtida arbeten kan innefatta optimering av BFP på Nvidias GPU-hårdvara med andra grafikkort än de två vi använde: RTX 5090 och 3090. Det finns mer avancerad hårdvara för större servrar. Dessa använder ett annat gränssnitt än PCIe, kallat NVLink, som enligt uppgift kan uppnå hastigheter upp till 900 GB/s nästan nio gånger snabbare än PCIe 5.0 x16 [20].

Litteraturförteckning

- [1] Open ran explained. hämtat : 2025-02-21, <https://www.nokia.com/networks/radio-access-networks/open-ran/open-ran-explained/>.
- [2] Anthony M. Bruno. Qam: The basics and beyond, 2023. hämtat : 2025-02-18, <https://www.cwnp.com/qam-basics/>.
- [3] Zhou Xia. What is qam?, 2022-04-02. hämtat : 2025-02-17, <https://info.support.huawei.com/info-finder/encyclopedia/en/QAM.html>.
- [4] Comparison of qam modulation types, November 22, 2016. hämtat : 2025-02-18, <https://www.gigabit-wireless.com/comparison-8-qam-16-qam-32-qam-64-qam-128-qam-256-qam-types/>.
- [5] Nvidia. Cuda c++ programming guide, hämtat : 2025-02-20. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#the-benefits-of-using-gpus>.
- [6] O-ran working group 4(open fronthaul interfaces wg) control, user and synchronization plane specification, 2024. hämtat : 2025-02-10, <https://specifications.o-ran.org/download?id=738>.
- [7] Mark Harris. An even easier introduction to cuda, 2017. hämtat : 2025-02-20, <https://developer.nvidia.com/blog/even-easier-introduction-cuda/>.

- [8] Burton R. Saltzberg. Comparison of single-carrier and multitone digital modulation for adsl applications, 2002. hämtat : 2025-02-17, <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=733485>.
- [9] Nvidia system profilerings verktyg. hämtat: 2025-04-25, <https://developer.nvidia.com/nsight-systems>.
- [10] Rtx3090 specifications. hämtat : 2025-06-20, <https://versus.com/en/nvidia-geforce-rtx-3090-vs-nvidia-geforce-rtx-5090>.
- [11] Rtx3090 specifications. hämtat : 2025-06-20, <https://pcisig.com/blog/evolution-pci-express-specification-its-sixth-generation-third-decade-and-s>
- [12] MiniCircuits. A primer on quadrature amplitude modulation (qam). hämtat: 2025-03-12, <https://blog.minicircuits.com/a-primer-on-quadrature-amplitude-modulation-qam/>.
- [13] What is 16-qam modulation?, 2022-07-19. hämtat: 2025-06-20, <https://www.everythingrf.com/community/what-is-16-qam-modulation>.
- [14] Larry. The basics of quadrature amplitude modulation(qam), 2024. hämtat : 2025-02-19, <https://info.support.huawei.com/info-finder/encyclopedia/en/QAM.html>.
- [15] NVIDIA. Cuda zone, hämtat : 2025-02-20. url<https://developer.nvidia.com/cuda-zone>.
- [16] Putt Sakdhnagool. Tutorial 01: Say hello to cuda. hämtat : 2025-02-20, <https://cuda-tutorial.readthedocs.io/en/latest/tutorials/tutorial01/>.
- [17] Cuda runtime api. hämtat : 2025-02-26, https://docs.nvidia.com/cuda/cuda-runtime-api/group__CUDART__EXECUTION.html.

- [18] Grafiskt remote verktyg. hämtat: 2025-04-15, <https://www.nomachine.com/>.
- [19] Tui remote verktyg. hämtat: 2025-04-15, <https://mobaxterm.mobatek.net/>.
- [20] Specification på nmlink. hämtat: 2025-04-15, <https://www.nvidia.com/en-us/data-center/h200/>.

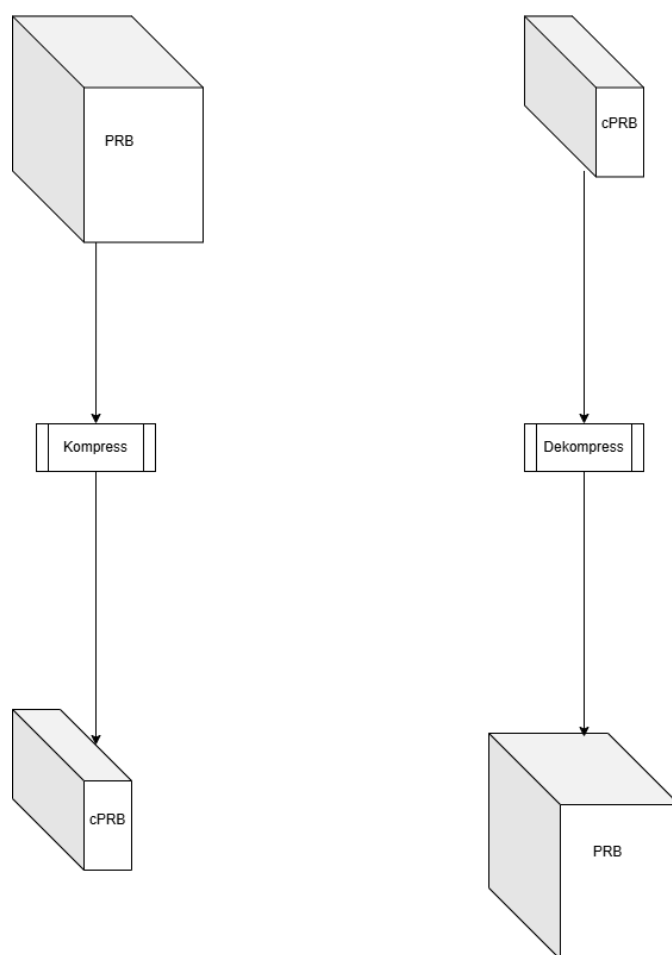
Bilagor

Bilaga A

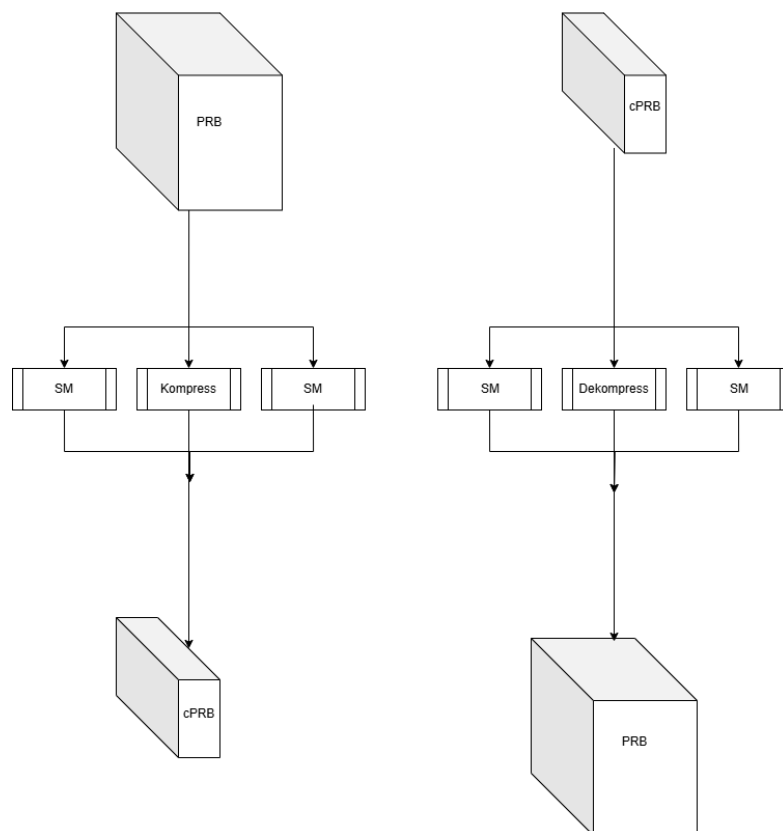
Bilagor

I Figur A.1 illustreras till vänster data i form av PRB som skickas till ett block som symboliserar en default SM på GPU som genererar en output på komprimerad data. Till höger är en liknande uppsättning men i omvänd ordning, komprimerad data avkomprimeras av en SM på GPU och blir tillbaka till original form av datan.

I Figur A.2 denna figur liknar fig. A.1 fast istället för en default stream så körs komprimering och dekomprimering på flera SM. Med flera SM så kan data delas upp och bearbetas på olika delar och på så sätt kan mer jobb utföras samtidigt.



Figur A.1: Visuell bild över program utan extra strömmar



Figur A.2: Visuell bild över program med extra strömmar

Bilaga B

Listningar

```
1
2  #include <stdio.h>
3  #include <math.h>
4  #include <cuda_runtime.h>
5  #include <stdlib.h>
6  #include <time.h>
7  #define NUMBER_OF_BLOCKS 2184
8  #define BLOCKSIZE 12
9  #define IQWIDTH 9
10 #define ITERATIONS 10000
11 #define DEBUG_DATA 0
12
13 typedef struct{
14     short Im[BLOCKSIZE];
15     short Q[BLOCKSIZE];
16     short max;
17     short min;
18     short sharedExponent;
19 }BFP;
20
21 typedef struct{
22     long d[3];
23     int d2[1];
24 }cBFP;
25
26 __device__ short arrMax(short *arr){
27     short max = arr[0];
28     for(int i = 1; i<BLOCKSIZE;i++){
29         if(max < arr[i])
30             max = arr[i];
31     }
32     return max;
33 }
```

```

34 __device__ short arrMin(short *arr){
35     short min = arr[0];
36     for(int i = 1; i<BLOCKSIZE;i++){
37         if(min > arr[i])
38             min = arr[i];
39     }
40     return min;
41 }
42 __device__ short max(short a, short b){
43     if(a >= b)
44         return a;
45     return b;
46 }
47 __device__ short min(short a, short b){
48     if(a <= b)
49         return a;
50     return b;
51 }
52 __global__ void compression(BFP *block){
53     for(int k = 0; k<NUMBER_OF_BLOCKS;k++){
54         block[k].max = max(arrMax(block[k].Im), arrMax(block[k].Q));
55         block[k].min = min(arrMin(block[k].Im), arrMin(block[k].Q));
56
57         short maxValue = max(block[k].max, abs(block[k].min) -1);
58
59         short rawExp = floor(log2(maxValue) +1);
60
61         short sharedExponent = max(rawExp - IQWIDTH +1, 0);
62
63         for( int i= 0 ; i<BLOCKSIZE;i++){
64             block[k].Im[i] = block[k].Im[i] >> sharedExponent;
65             block[k].Q[i] = block[k].Q[i] >> sharedExponent;
66         }
67         block[k].sharedExponent = sharedExponent;
68     }
69 }
70 }
71 __global__ void decompress(BFP *block){
72     for(int k = 0; k<NUMBER_OF_BLOCKS;k++){
73         for( int i= 0 ; i<BLOCKSIZE;i++){
74             block[k].Im[i] = block[k].Im[i] << block[k].sharedExponent;
75             block[k].Q[i] = block[k].Q[i] << block[k].sharedExponent;
76         }
77     }
78 }
79
80 int main(){
81     BFP Block[NUMBER_OF_BLOCKS];
82     BFP *d_Block;

```

```

83     cudaMalloc((void **)&d_Block, sizeof(BFP) * NUMBER_OF_BLOCKS);
84
85     for(int k = 0; k< NUMBER_OF_BLOCKS; k++){
86         for(int i = 0 ; i<BLOCKSIZE;i++){
87             int tmp;
88             scanf("%d", &tmp);
89             Block[k].Im[i] = (tmp & 0xFFFF0000) >> 16;
90             Block[k].Q[i] = tmp & 0x0000FFFF;
91         }
92     }
93
94     cudaMemcpy(d_Block, Block, sizeof(BFP) * NUMBER_OF_BLOCKS, cudaMemcpyHostToDevice);
95
96     if(DEBUG_DATA){
97         printf("input data\n");
98         for(int k = 0; k<NUMBER_OF_BLOCKS;k++){
99             for(int i = 0 ; i < BLOCKSIZE;i++){
100                 printf("data row Im, %d ,Q %d \n", Block[k].Im[i], Block[k].Q[i] );
101             }
102             //printf("shared exponent is : %d\n", Block[k].sharedExponent);
103             puts("");
104         }
105     }
106
107     struct timespec start1,end1, start2, end2;
108     long compTimes[ITERATIONS], reverseTimes[ITERATIONS];
109
110     for ( int j = 0 ; j < ITERATIONS;j++){
111         clock_gettime(CLOCK_MONOTONIC, &start1);
112         compression<<<1,1>>>(d_Block);
113         cudaDeviceSynchronize();
114         clock_gettime(CLOCK_MONOTONIC, &end1);
115         compTimes[j] = (end1.tv_sec - start1.tv_sec)*1000000000L +(end1.tv_nsec -start1.tv_nsec);
116
117
118         clock_gettime(CLOCK_MONOTONIC, &start2);
119         decompress<<<1,1>>>(d_Block);
120         cudaDeviceSynchronize();
121         clock_gettime(CLOCK_MONOTONIC, &end2);
122         reverseTimes[j] = (end2.tv_sec - start2.tv_sec)*1000000000L + (end2.tv_nsec -start2.tv_nsec);
123     }
124
125     cudaMemcpy(Block, d_Block, sizeof(BFP) * NUMBER_OF_BLOCKS, cudaMemcpyDeviceToHost);
126
127     if(DEBUG_DATA){
128         printf("after de compression\n");
129         for(int k = 0; k<NUMBER_OF_BLOCKS;k++){
130             for(int i = 0 ; i < BLOCKSIZE;i++){
131                 printf("data row Im, %d ,Q %d \n", Block[k].Im[i], Block[k].Q[i] );

```

```

132         }
133         printf("shared exponent is : %d\n", Block[k].sharedExponent);
134         puts("");
135     }
136 }
137 puts("Single Thread GPU");
138 for(int i = 0; i < ITERATIONS; i++){
139     printf("%ld ", compTimes[i]);
140 }
141 puts("");
142 for(int i = 0; i < ITERATIONS; i++){
143     printf("%ld ", reverseTimes[i]);
144 }
145 cudaFree(d_Block);
146 }

```

Listning B.1: Seriell exekvering på GPU

```

1  #include <stdio.h>
2  #include <limits.h>
3  #include <math.h>
4  #include <cuda_runtime.h>
5  #include <stdlib.h>
6  #include <time.h>
7  #define NUMBER_OF_BLOCKS 2184
8  #define ROWS 12
9  #define COLUMNS 2
10 #define IQWIDTH 9
11 #define ITERATIONS 10000
12 #define DEBUG_DATA 0
13
14 typedef struct{
15     short data[ROWS][COLUMNS];
16     short max;
17     short min;
18     short sharedExponent;
19 }BFP;
20
21 typedef struct{
22     long d[3];
23     int d2[1];
24 }cBFP;
25
26 __device__ short arrMax(short arr[ROWS][COLUMNS]){
27     short max = SHRT_MIN;
28     for(int i = 0; i < ROWS; i++){
29         for(int j = 0; j < COLUMNS; j++){

```

```

30         if(max < arr[i][j])
31             max = arr[i][j];
32     }
33 }
34 return max;
35 }
36 __device__ short arrMin(short arr[ROWS][COLUMNS]){
37     short min = SHRT_MAX;
38     for(int i = 0; i<ROWS;i++){
39         for(int j = 0; j<COLUMNS;j++){
40             if(min > arr[i][j])
41                 min = arr[i][j];
42         }
43     }
44     return min;
45 }
46 __device__ short max(short a, short b){
47     if(a >= b)
48         return a;
49     return b;
50 }
51 __device__ short min(short a, short b){
52     if(a <= b)
53         return a;
54     return b;
55 }
56 __global__ void compression(BFP *block){
57     int index = threadIdx.x;
58     int colum = threadIdx.y;
59     for(int k = 0; k<NUMBER_OF_BLOCKS;k++){
60         if(index== 0){
61             block[k].max = arrMax(block[k].data);
62             block[k].min = arrMin(block[k].data);
63
64             short maxValue = max(block[k].max, abs(block[k].min) -1);
65
66             short rawExp = floor(log2(maxValue) +1);
67
68             short sharedExponent = max(rawExp - IQWIDTH +1, 0);
69             block[k].sharedExponent = sharedExponent;
70         }
71         __syncthreads();
72         block[k].data[index][colum] = block[k].data[index][colum] >> block[k].sharedExponent;
73     }
74 }
75 __global__ void decompress(BFP *block){
76     int index = threadIdx.x;
77     int colum = threadIdx.y;
78     for(int k = 0; k<NUMBER_OF_BLOCKS;k++){

```

```

79         block[k].data[i][column] = block[k].data[i][column] << block[k].sharedExponent;
80     }
81
82     int main(){
83         BFP Block[NUMBER_OF_BLOCKS];
84         //cBFP compBlock[NUMBER_OF_BLOCKS];
85         BFP *d_Block;
86         cudaMalloc((void **)&d_Block, sizeof(BFP) * NUMBER_OF_BLOCKS);
87
88         for(int k = 0; k< NUMBER_OF_BLOCKS; k++){
89             for(int i = 0 ; i<ROWS;i++){
90                 int tmp;
91                 scanf("%d", &tmp);
92                 Block[k].data[i][0] = (tmp & 0xFFFF0000) >> 16;
93                 Block[k].data[i][1] = tmp & 0x0000FFFF;
94             }
95         }
96
97         cudaMemcpy(d_Block, Block, sizeof(BFP) * NUMBER_OF_BLOCKS, cudaMemcpyHostToDevice);
98
99         if(DEBUG_DATA){
100             printf("input data\n");
101             for(int k = 0; k<NUMBER_OF_BLOCKS;k++){
102                 for(int i = 0 ; i < ROWS;i++){
103                     printf("data row %d, %d ,Q %d \n", Block[k].data[i][0], Block[k].data[i][1] );
104                 }
105                 //printf("shared exponent is : %d\n", Block[k].sharedExponent);
106                 puts("");
107             }
108         }
109         struct timespec start1,end1, start2, end2;
110         long compTimes[ITERATIONS], reverseTimes[ITERATIONS];
111
112         for ( int j = 0 ; j < ITERATIONS;j++){
113             clock_gettime(CLOCK_MONOTONIC, &start1);
114             compress<<<1,dim3(12,2,1)>>>(d_Block);
115             cudaDeviceSynchronize();
116             clock_gettime(CLOCK_MONOTONIC, &end1);
117             compTimes[j]= (end1.tv_sec - start1.tv_sec)*1000000000L +(end1.tv_nsec -start1.tv_nsec);
118
119
120             clock_gettime(CLOCK_MONOTONIC, &start2);
121             decompress<<<1,dim3(12,2,1)>>>(d_Block);
122             cudaDeviceSynchronize();
123             clock_gettime(CLOCK_MONOTONIC, &end2);
124             reverseTimes[j] = (end2.tv_sec - start2.tv_sec)*1000000000L + (end2.tv_nsec -start2.tv_nsec);
125         }
126
127         cudaMemcpy(Block, d_Block, sizeof(BFP) * NUMBER_OF_BLOCKS, cudaMemcpyDeviceToHost);

```

```

128
129     if(DEBUG_DATA){
130         printf("after de compression\n");
131         for(int k = 0; k<NUMBER_OF_BLOCKS;k++){
132             for(int i = 0 ; i < ROWS;i++){
133                 printf("data row %d, %d, %d \n", Block[k].data[i][0], Block[k].data[i][1] );
134             }
135             printf("shared exponent is : %d\n", Block[k].sharedExponent);
136             puts("");
137         }
138     }
139     puts("GPU carries in parallel 24 th");
140     for(int i = 0;i<ITERATIONS;i++){
141         printf("%ld ", compTimes[i]);
142     }
143     puts("");
144     for(int i = 0;i<ITERATIONS;i++){
145         printf("%ld ", reverseTimes[i]);
146     }
147     cudaFree(d_Block);
148 }

```

Listning B.2: GPU parallell i ett PRB

```

1  #include <stdio.h>
2  #include <limits.h>
3  #include <math.h>
4  #include <cuda_runtime.h>
5  #include <stdlib.h>
6  #include <time.h>
7  #define NUMBER_OF_BLOCKS 2184
8  #define ROWS 12
9  #define COLUMNS 2
10 #define IQWIDTH 9
11 #define ITERATIONS 10000
12 #define DEBUG_DATA 0
13
14 typedef struct{
15     short data[ROWS][COLUMNS];
16     short max;
17     short min;
18     short sharedExponent;
19 }BFP;
20
21 typedef struct{
22     long d[3];
23     int d2[1];

```

```

24 }cBFP;
25
26 __device__ short arrMax(short arr[ROWS][COLUMNS]){
27     short max = SHRT_MIN;
28     for(int i = 0; i<ROWS;i++){
29         for(int j = 0; j<COLUMNS;j++){
30             if(max < arr[i][j])
31                 max = arr[i][j];
32         }
33     }
34     return max;
35 }
36 __device__ short arrMin(short arr[ROWS][COLUMNS]){
37     short min = SHRT_MAX;
38     for(int i = 0; i<ROWS;i++){
39         for(int j = 0; j<COLUMNS;j++){
40             if(min > arr[i][j])
41                 min = arr[i][j];
42         }
43     }
44     return min;
45 }
46 __device__ short max(short a, short b){
47     if(a >= b)
48         return a;
49     return b;
50 }
51 __device__ short min(short a, short b){
52     if(a <= b)
53         return a;
54     return b;
55 }
56 __global__ void compression(BFP *block){
57     int index = threadIdx.x;
58     int colum = threadIdx.y;
59     int PFB_index = blockIdx.x * blockDim.z + threadIdx.z;
60
61     if(index== 0){
62         block[PFB_index].max = arrMax(block[PFB_index].data);
63         block[PFB_index].min = arrMin(block[PFB_index].data);
64
65         short maxValue = max(block[PFB_index].max, abs(block[PFB_index].min) -1);
66
67         short rawExp = floor(log2(maxValue) +1);
68
69         short SharedExponent = max(rawExp - IQWIDTH +1, 0);
70         block[PFB_index].sharedExponent = sharedExponent;
71     }
72     __syncthreads();

```



```

73     for(int i = 0; i<NUMBER_OF_ROWS;i++){
74         block[PFB_index].data[i][0] = block[PFB_index].data[i][0] >>
75         block[PFB_index].sharedExponent;
76         block[PFB_index].data[i][1] = block[PFB_index].data[i][1] >>
77         block[PFB_index].sharedExponent;
78     }
79 }
80 __global__ void decompress(BFP *block){
81     int index = threadIdx.x;
82     int colum = threadIdx.y;
83     int PFB_index = blockIdx.x * blockDim.z + threadIdx.z;
84
85     for(int i = 0; i<NUMBER_OF_ROWS;i++){
86         block[PFB_index].data[i][0] = block[PFB_index].data[i][0] <<
87         block[PFB_index].sharedExponent;
88         block[PFB_index].data[i][1] = block[PFB_index].data[i][1] <<
89         block[PFB_index].sharedExponent;
90     }
91 }
92
93 int main(){
94     BFP Block[NUMBER_OF_BLOCKS];
95     //cBFP compBlock[NUMBER_OF_BLOCKS];
96     BFP *d_Block;
97     cudaMalloc((void **)&d_Block, sizeof(BFP) * NUMBER_OF_BLOCKS);
98
99     for(int k = 0; k< NUMBER_OF_BLOCKS; k++){
100         for(int i = 0 ; i<ROWS;i++){
101             int tmp;
102             scanf("%d", &tmp);
103             Block[k].data[i][0] = (tmp & 0xFFFF0000) >> 16;
104             Block[k].data[i][1] = tmp & 0x0000FFFF;
105         }
106     }
107
108
109
110     if(DEBUG_DATA){
111         printf("input data\n");
112         for(int k = 0; k<NUMBER_OF_BLOCKS;k++){
113             for(int i = 0 ; i < ROWS;i++){
114                 printf("data row %d, %d ,Q %d \n", Block[k].data[i][0],
115                     Block[k].data[i][1] );
116             }
117             puts("");
118         }
119     }
120     struct timespec start1,end1, start2, end2;
121     long compTimes[ITERATIONS], reverseTimes[ITERATIONS];

```

```

122
123     for ( int j = 0 ; j < ITERATIONS;j++){
124         clock_gettime(CLOCK_MONOTONIC, &start1);
125         cudaMemcpy(d_Block, Block, sizeof(BFP) * NUMBER_OF_BLOCKS, cudaMemcpyHostToDevice);
126         compression<<< 1,2184>>>(d_Block);
127         cudaMemcpy(Block, d_Block, sizeof(BFP) * NUMBER_OF_BLOCKS, cudaMemcpyDeviceToHost);
128         cudaDeviceSynchronize();
129         clock_gettime(CLOCK_MONOTONIC, &end1);
130         compTimes[j]= (end1.tv_sec - start1.tv_sec)*1000000000L +
131         (end1.tv_nsec -start1.tv_nsec);
132
133
134         clock_gettime(CLOCK_MONOTONIC, &start2);
135         cudaMemcpy(d_Block, Block, sizeof(BFP) * NUMBER_OF_BLOCKS, cudaMemcpyHostToDevice);
136         decompress<<<1,2184>>>(d_Block);
137         cudaMemcpy(Block, d_Block, sizeof(BFP) * NUMBER_OF_BLOCKS, cudaMemcpyDeviceToHost);
138         cudaDeviceSynchronize();
139         clock_gettime(CLOCK_MONOTONIC, &end2);
140         reverseTimes[j] = (end2.tv_sec - start2.tv_sec)*1000000000L +
141         (end2.tv_nsec -start2.tv_nsec);
142     }
143
144
145
146     if(DEBUG_DATA){
147         printf("after de compression\n");
148         for(int k = 0; k<NUMBER_OF_BLOCKS;k++){
149             for(int i = 0 ; i < ROWS;i++){
150                 printf("data row %d, %d ,Q %d \n", Block[k].data[i][0],
151                 Block[k].data[i][1] );
152             }
153             printf("shared exponent is : %d\n", Block[k].sharedExponent);
154             puts("");
155         }
156     }
157     puts("GPU full parallel 2184 threads");
158     for(int i = 0;i<ITERATIONS;i++){
159         printf("%ld ", compTimes[i]);
160     }
161     puts("");
162     for(int i = 0;i<ITERATIONS;i++){
163         printf("%ld ", reverseTimes[i]);
164     }
165     cudaFree(d_Block);
166 }
167

```

Listning B.3: GPU parallell över PRB

```

1  #include <stdio.h>
2  #include <string.h>
3  #include <limits.h>
4  #include <math.h>
5  #include <cuda_runtime.h>
6  #include <stdlib.h>
7  #include <time.h>
8  #include <unistd.h>
9  #define ROWS 12
10 #define COLUMNS 2
11 #define ZTHREADS 4
12 #define IQWIDTH 9
13 #define ITERATIONS 280
14 #define DEBUG_DATA 1
15 #define NUMBER_OF_SYMB 14
16
17
18 typedef struct{
19     short data[ROWS][COLUMNS];
20     short sharedExponent;
21 }BFP;
22
23 typedef struct{
24     int data[8];
25 }cBFP;
26
27 __global__ void varm_up(){
28     int var = 5 * threadIdx.x;
29     var *= 10;
30 }
31
32 __device__ short arrMax(short arr[ROWS][COLUMNS]){
33     short max = SHRT_MIN;
34     for(int i = 0; i<ROWS;i++){
35         for(int j = 0; j<COLUMNS;j++){
36             if(max < arr[i][j])
37                 max = arr[i][j];
38         }
39     }
40     return max;
41 }
42 __device__ short arrMin(short arr[ROWS][COLUMNS]){
43     short min = SHRT_MAX;
44     for(int i = 0; i<ROWS;i++){
45         for(int j = 0; j<COLUMNS;j++){
46             if(min > arr[i][j])
47                 min = arr[i][j];
48         }

```

```

49     }
50     return min;
51 }
52 __device__ short max(short a, short b){
53     if(a >= b)
54         return a;
55     return b;
56 }
57 __device__ short min(short a, short b){
58     if(a <= b)
59         return a;
60     return b;
61 }
62 __global__ void compression(BFP *block){
63     int index = threadIdx.x; //0-11
64     int column = threadIdx.y; //0-1
65     int PFB_index = blockIdx.x * blockDim.z + threadIdx.z;
66     //blockIdx.x= 0-363, blockDim.z=6, threadIdx.z= 0-5
67
68     if((index== 0) && (column == 0)){
69         short maxN = arrMax(block[PFB_index].data);
70         short minN = arrMin(block[PFB_index].data);
71
72         short maxValue = max(maxN , abs(minN) -1);
73
74         short rawExp = floor(log2(maxValue) +1);
75
76         block[PFB_index].sharedExponent = max(rawExp - IQWIDTH +1, 0);
77     }
78     __syncthreads();
79
80     block[PFB_index].data[index][column] = (block[PFB_index].data[index][column] >>
81     block[PFB_index].sharedExponent);
82
83 }
84 __global__ void decompress(BFP *block){
85     int index = threadIdx.x;
86     int column = threadIdx.y;
87     int PFB_index = blockIdx.x * blockDim.z + threadIdx.z;
88
89     block[PFB_index].data[index][column] = (block[PFB_index].data[index][column] <<
90     block[PFB_index].sharedExponent);
91 }
92 void readData(BFP *Block, FILE *FP, int NUMBER_OF_BLOCKS){
93     for(int k = 0; k< NUMBER_OF_BLOCKS; k++){
94         for(int i = 0 ; i<ROWS;i++){
95             int tmp;
96             fprintf(stderr,"%d " ,fscanf(FP,"%d", &tmp));
97             Block[k].data[i][0] = (tmp & 0xFFFF0000) >> 16;

```

```

98         Block[k].data[i][1] = tmp & 0x0000FFFF;
99     }
100 }
101 }
102 int partFunc(int total, int splits){
103     if(splits == 1)
104         return 0;
105     return total /splits;
106 }
107
108 int main(int argc, char **argv){
109
110     if(argc < 2 || argc > 4)
111         return -1;
112
113     int NUMBER_OF_BLOCKS = 2184;
114     int NUMBER_OF_STREAMS = 14;
115     int CUDA_BLOCK = 546 *NUMBER_OF_SYMB;
116     bool isHighbandMode = true;
117     if(argc == 4 && strcmp(argv[3], "lowband") == 0){ //prepares # of block, HB or LB
118         printf("running lowband \n\n");
119         NUMBER_OF_BLOCKS = 400;
120         NUMBER_OF_STREAMS = 7;
121         CUDA_BLOCK = 100 * NUMBER_OF_SYMB;
122         isHighbandMode = false;
123     }
124     if(atoi(argv[1])!= 0)
125         NUMBER_OF_STREAMS = atoi(argv[1]);
126     CUDA_BLOCK /= NUMBER_OF_STREAMS;
127
128     int NUMBER_OF_MAST = 1;
129     if(atoi(argv[2])!= 0) //prepares # of masts
130         NUMBER_OF_MAST = atoi(argv[2]);
131
132     BFP *Block;
133     BFP *d_Block;
134
135     fprintf(stderr,"%d", cudaMallocHost((void **)&Block, sizeof(BFP) * NUMBER_OF_BLOCKS *
136     NUMBER_OF_SYMB * NUMBER_OF_MAST) != cudaSuccess);
137     return -1;
138     if(cudaMalloc((void **)&d_Block, sizeof(BFP) * NUMBER_OF_BLOCKS * NUMBER_OF_SYMB *
139     NUMBER_OF_MAST) != cudaSuccess)
140         return -1;
141
142
143     cudaMemcpy(d_Block,Block , sizeof(BFP) * NUMBER_OF_BLOCKS*
144     NUMBER_OF_SYMB, cudaMemcpyHostToDevice);
145
146     struct timespec start1,end1;

```

```

147     long compTimes[ITERATIONS];
148     cudaStream_t streams[NUMBER_OF_STREAMS * NUMBER_OF_MAST];
149     int DataAmount = sizeof(BFP) * NUMBER_OF_BLOCKS * NUMBER_OF_SYMB /
150     NUMBER_OF_STREAMS;
151
152     for(int i = 0; i < NUMBER_OF_STREAMS * NUMBER_OF_MAST; i++){
153         cudaStreamCreate(&(streams[i]));
154     }
155     FILE *FP = NULL;
156     FP = fopen("BetterData.txt", "r");
157     for(int i = 0 ; i < ITERATIONS; i++)
158         compTimes[i] = 0;
159     for(int i = 0; i < 1000; i++)
160         varm_up<<<1024,1024>>>();
161     cudaLaunchKernel((void *)compression, 1, 1, NULL, 0, 0);
162     cudaDeviceSynchronize();
163     for ( int j = 0 ; j < ITERATIONS / NUMBER_OF_SYMB; j++){
164         for(int k = 0; k < NUMBER_OF_SYMB * NUMBER_OF_MAST; k++){
165             readData(Block + (NUMBER_OF_BLOCKS * k), FP , NUMBER_OF_BLOCKS);
166
167             if(DEBUG_DATA){
168                 for(int k = 0; k < NUMBER_OF_SYMB * NUMBER_OF_BLOCKS; k++){
169                     for(int n = 0; n < 12; n++){
170                         fprintf(stderr, "%d %d ", Block[k].data[n][0], Block[k].data[n][1]);
171                     }
172                     fprintf(stderr, "\n");
173                 }
174             }
175             clock_gettime(CLOCK_MONOTONIC, &start1);
176             for(int k = 0 ; k < NUMBER_OF_STREAMS * NUMBER_OF_MAST; k++){
177                 if (j * NUMBER_OF_STREAMS + k == 100)
178                     break;
179                 cudaMemcpyAsync(d_Block + (NUMBER_OF_BLOCKS * k), Block +
180                 (NUMBER_OF_BLOCKS *
181                 k), DataAmount , cudaMemcpyHostToDevice, streams[k]);
182             }
183
184
185             for(int k = 0 ; k < NUMBER_OF_STREAMS * NUMBER_OF_MAST; k++){
186                 compression<<<CUDA_BLOCK , dim3(12,2,ZTHREADS), 0, streams[k]>>>
187                 (d_Block + (NUMBER_OF_BLOCKS * k));
188
189                 for(int k = 0 ; k < NUMBER_OF_STREAMS * NUMBER_OF_MAST; k++){
190                     cudaMemcpyAsync(Block + (NUMBER_OF_BLOCKS * k), d_Block +
191                     (NUMBER_OF_BLOCKS *
192                     k), DataAmount, cudaMemcpyDeviceToHost, streams[k]);
193                 }
194                 cudaDeviceSynchronize();
195                 clock_gettime(CLOCK_MONOTONIC, &end1);
196                 compTimes[j] = (end1.tv_sec - start1.tv_sec)*1000000000L +

```

```

196         (end1.tv_nsec -start1.tv_nsec);
197     if(DEBUG_DATA){
198         cudaDeviceSynchronize();
199         for(int k = 0; k<NUMBER_OF_BLOCKS*NUMBER_OF_SYMB;k++){
200             for(int n = 0; n<12;n++){
201                 printf("%d %d ", Block[k].data[n][0], Block[k].data[n][1]);
202             }
203         }
204     }
205 }
206
207 if(!DEBUG_DATA){
208     printf("GPU full parallel %dx12x2x2 ~ %d threads per SM, number of PRB per SM: %d,
209     number of SM: %d, number of cell %d \n",CUDA_BLOCK, CUDA_BLOCK*96,
210     NUMBER_OF_BLOCKS, NUMBER_OF_STREAMS, NUMBER_OF_MAST);
211     printf("total amount of threads: %d \n", NUMBER_OF_MAST*NUMBER_OF_STREAMS * CUDA_BLOCK*48);
212     printf("total amount of cores used: %d of a total of for RTX3090 (10496) and RTX5090
213     (21760) \n", NUMBER_OF_STREAMS * NUMBER_OF_MAST * 96);
214     int tmp = isHighbandMode ? 35000 * NUMBER_OF_SYMB : 72000 * NUMBER_OF_SYMB;
215
216     for(int i = 0;i<ITERATIONS / NUMBER_OF_SYMB;i++){
217         printf("sum: %ld , max: %d\n", compTimes[i], tmp);
218     }
219 }
220
221 puts("");
222 fclose(FP);
223 cudaFree(d_Block);
224 cudaFree(Block);
225 for(int i = 0;i < NUMBER_OF_STREAMS * NUMBER_OF_MAST;i++)
226     cudaStreamDestroy(streams[i]);
227
228 }

```

Listning B.4: GPU helt parallell